

2025-겨울학기 Dash Lab 학부연구인턴

Pruning&Quantization



중앙대학교 전자전기공학부 | 홍동의

Index

목차소개

- 01** Dataset & Model
- 02** About Pruning
- 03** Pruning simulation
- 04** About Quantization
- 05** Quantization simulation

01 Dataset & Model



chenyaof / pytorch-cifar-models

[pytorch-cifar-models/pytorch_cifar_models at master · chenyaof/pytorch-cifar-models](https://github.com/chenyaof/pytorch-cifar-models)

CIFAR-10



Input size: 3x32x32
Train size: 45000
Val size: 5000
Test size: 10000
Batch_size: 128

VGG-16



Convolution layer: 13개
FC layer: 3개

Convolution layer에 batchnormalization 추가
FC layer에 dropout 추가

Pretrained weight



Top-1 Acc : 94.16%
Top-5 Acc : 99.71%
Parameters : 15.25M
Madds : 313.73M

01 Dataset & Model

```
features.0.weight: torch.Size([64, 3, 3, 3])
features.3.weight: torch.Size([64, 64, 3, 3])
features.7.weight: torch.Size([128, 64, 3, 3])
features.10.weight: torch.Size([128, 128, 3, 3])
features.14.weight: torch.Size([256, 128, 3, 3])
features.17.weight: torch.Size([256, 256, 3, 3])
features.20.weight: torch.Size([256, 256, 3, 3])
features.24.weight: torch.Size([512, 256, 3, 3])
features.27.weight: torch.Size([512, 512, 3, 3])
features.30.weight: torch.Size([512, 512, 3, 3])
features.34.weight: torch.Size([512, 512, 3, 3])
features.37.weight: torch.Size([512, 512, 3, 3])
features.40.weight: torch.Size([512, 512, 3, 3])
classifier.0.weight: torch.Size([512, 512])
classifier.3.weight: torch.Size([512, 512])
classifier.6.weight: torch.Size([10, 512])
```

```
rcorrect,acc=mm.Test(model, test_DL, DEVICE)
print(f"Test accuracy: {rcorrect}/{len(test_DL.dataset)} ({acc} %)")
```

✓ 23.4s

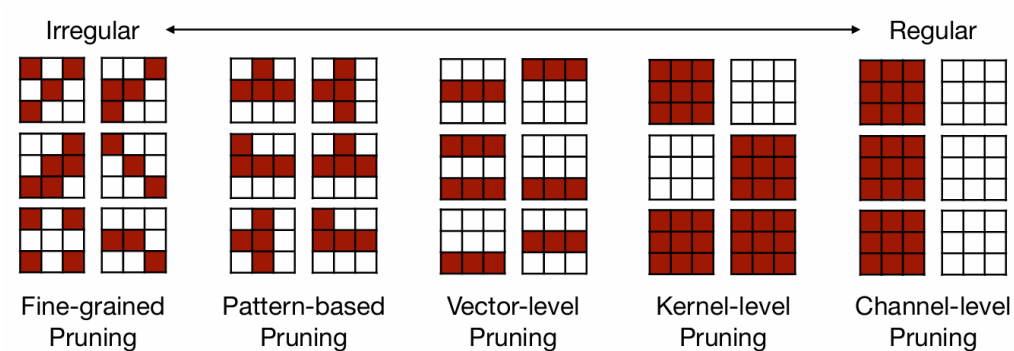
Test accuracy: 9416/10000 (94.2 %)

02 About Pruning



Pruning을 하기 위해서 설정해야 하는 3가지 항목.

Granularity

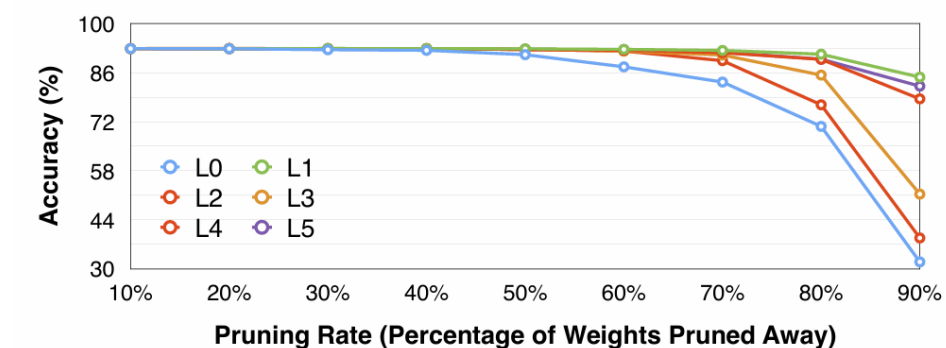


Criterion

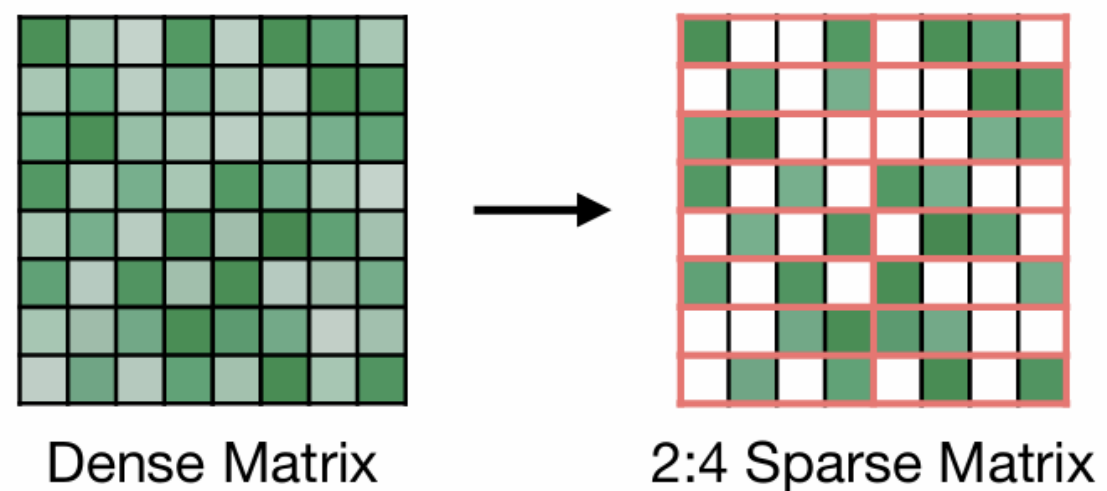
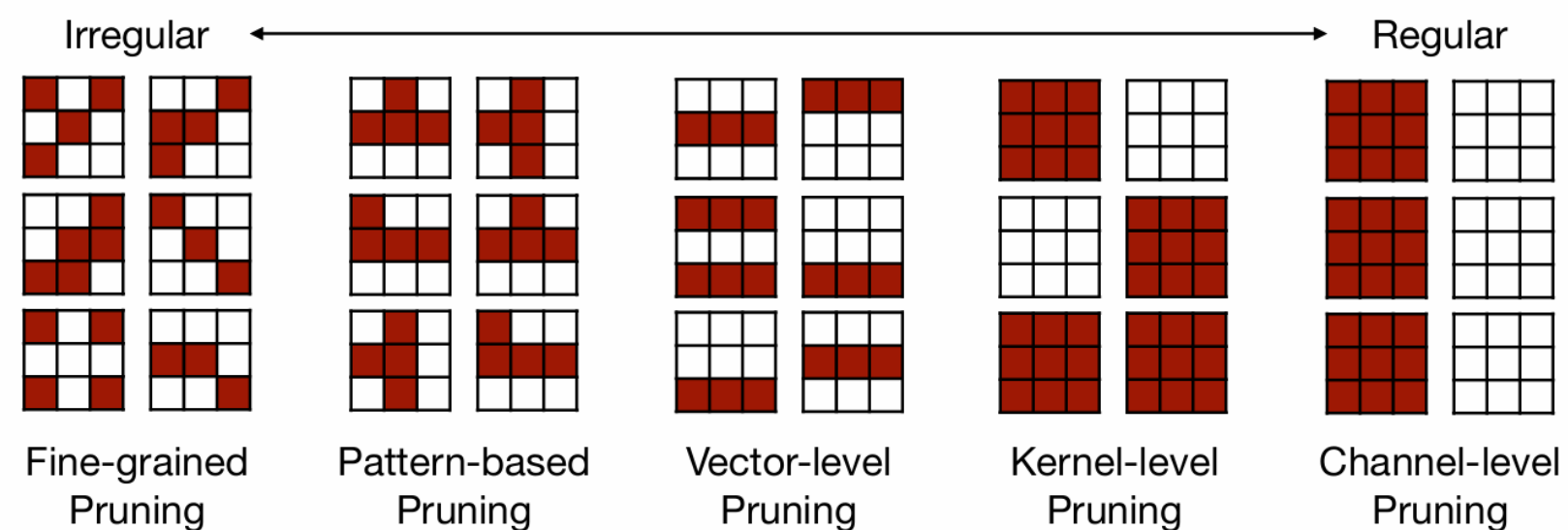
$$Importance = \sum_{i \in S} |w_i|$$

$$Importance = \sqrt{\sum_{i \in S} |w_i|^2}$$

Pruning Ratio



02 About Pruning -Granularity



요약

- Irregular 할 수록 많은 데이터 프루닝 가능
(같은 비율에 대해 acc drop 적음)
- But, 저장해야 하는 index가 많아지고, Hardware 차원에서 가속에 어려움이 있음

02 About Pruning –Criterion

- **Magnitude based Pruning**

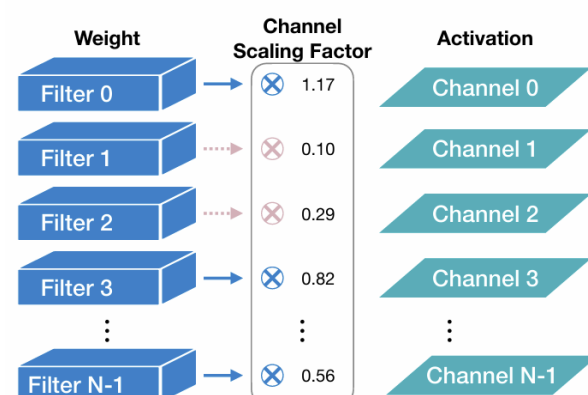
- **L1 Magnitude**

$$Importance = \sum_{i \in S} |w_i|$$

- **L2 Magnitude**

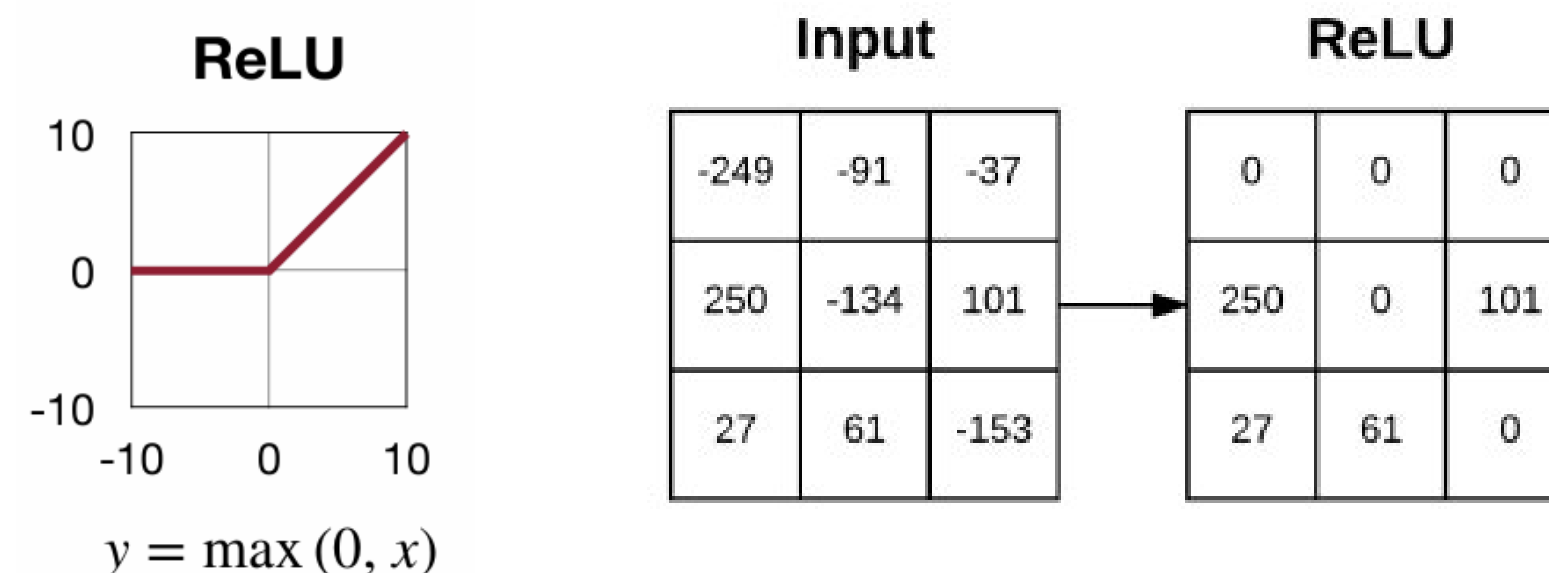
$$Importance = \sqrt{\sum_{i \in S} |w_i|^2}$$

- **Scaling-based Pruning**



$$z_o = \gamma \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \beta$$

- **Percentage-of-Zero-based Pruning (APoZ)**

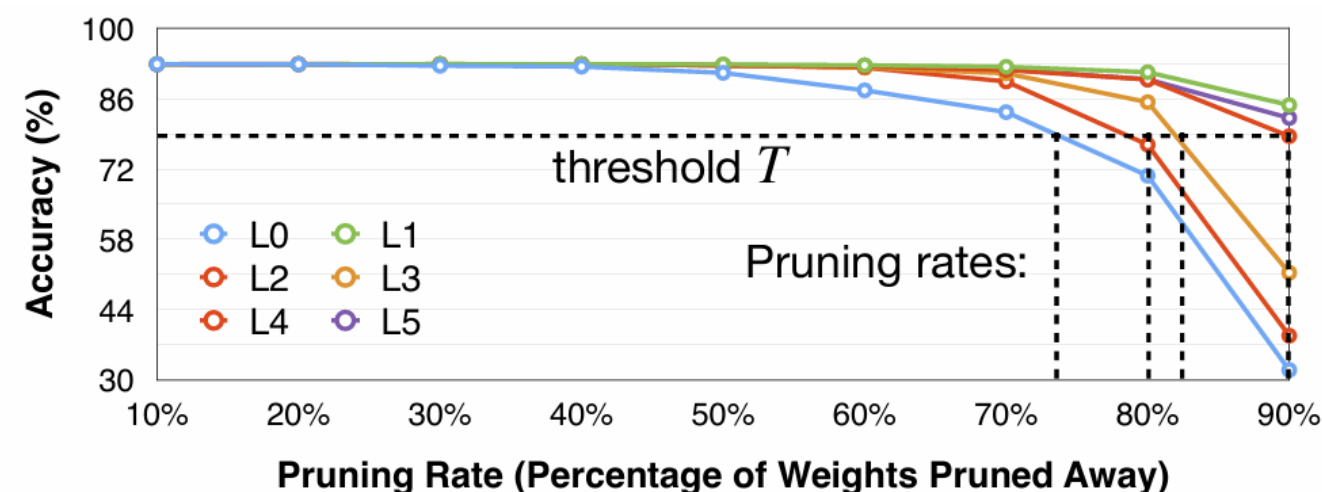


- **Second-Order-based Pruning**

- **Regression-based Pruning**

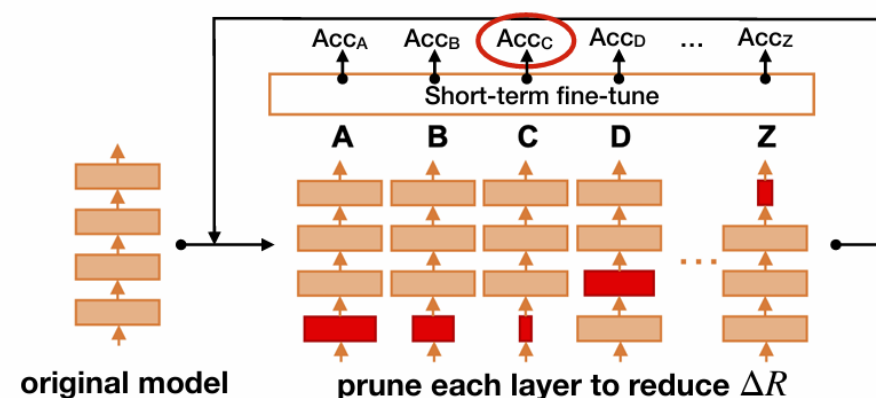
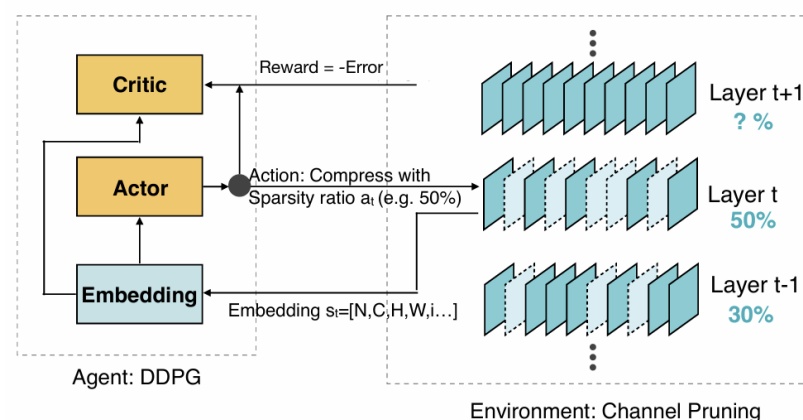
02 About Pruning – Ratio

- Analyze the sensitivity of each layer



- 각 Layer에 다양한 pruning rate을 적용했을 때 accuracy drop을 기록
 - 쉬움
 - Layer간 상호작용을 고려하지 않음

- Automatic Pruning
 - AutoML for Model Compression
 - NetAdapt



- AMC: 강화학습을 이용하여 layer별 ratio를 구함
- NetAdapt: 각 layer 마다 똑같은 latency drop을 만드는 ratio적용
 - Short-term finetuning을 하여 acc가 가장 높은 경우를 선택
 - latency의 saturation이 일어날 때 까지 반복
 - Long-term finetuning

02 About Pruning – Regularization

- **L1 normalization**

$$Loss = data_loss + \lambda |w| \quad \longrightarrow \quad \frac{dLoss}{dw} = \frac{d(data_loss)}{dw} + \lambda$$

- Weight 크기에 상관없이 같은 크기의 gradient
- 크기가 작은 weight 는 계속 작아지므로 0이 됨
- 크기가 큰 weight만 남기는 pruning 효과

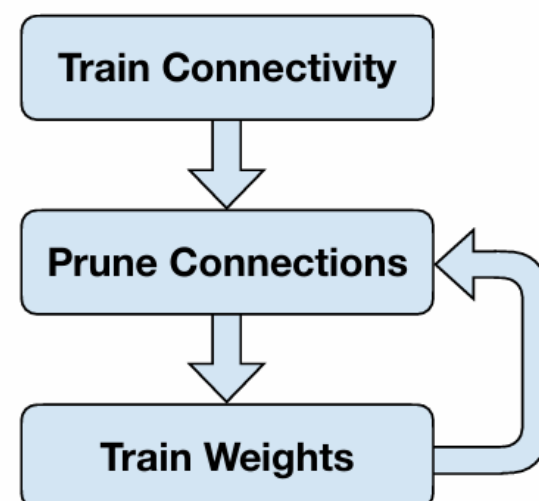
- **L2 normalization**

$$Loss = data_loss + \frac{1}{2} \lambda |w|^2 \quad \longrightarrow \quad \frac{dLoss}{dw} = \frac{d(data_loss)}{dw} + \lambda w$$

- Weight 크기에 비례하는 gradient
- Weight의 값이 크면 gradient가 크게 바뀜
- Weight 값이 작으면 gradient가 많이 안 바뀜
- 모든 값이 고르게 되는 효과

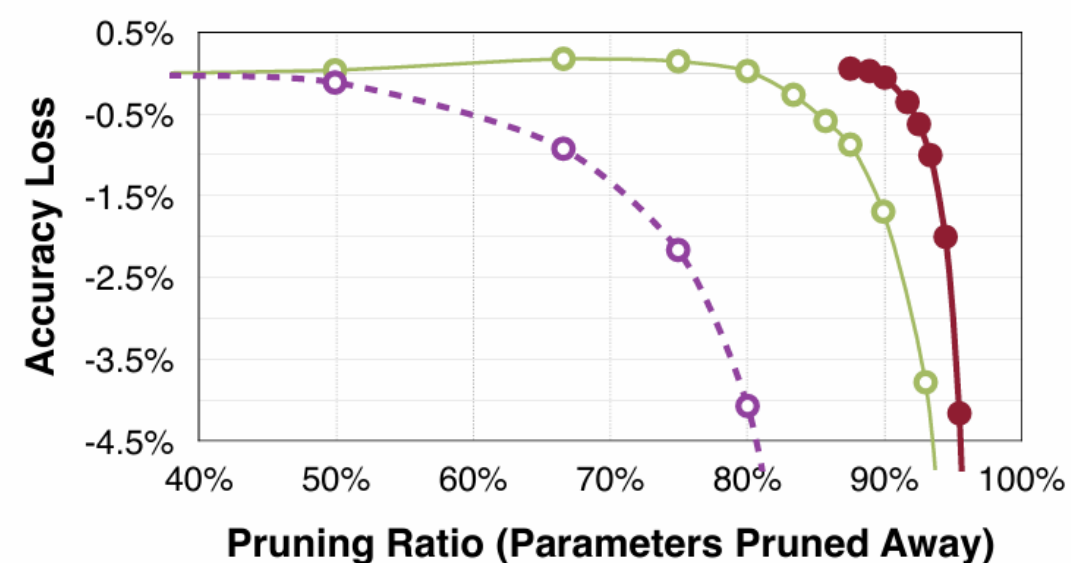
02 About Pruning – Fine Tuning

• Iterative Pruning



- Pruning이 된 모델을 재학습 하는 과정
- Pruning ratio를 높이기 위해 재학습 된 가중치를 다시 pruning
- 같은 비율에 대해 Iterative pruning을 진행하는 것이 한번에 pruning 후 finetuning 하는거에 비해 accuracy drop이 적음

○ Pruning ○ Pruning+Finetuing ● Iterative Pruning and Finetuing



02 About Pruning – Fine Tuning

- Pruning Mask

0.85	-0.12	1.20	0.05
-0.03	0.74	-0.91	-0.25
0.45	0.01	-0.62	1.55
-0.19	1.05	0.04	0.33



1	0	1	0
0	1	1	0
1	0	1	1
0	1	0	1



0.85	0	1.20	0
0	0.74	-0.91	0
0.45	0	-0.62	1.55
0	1.05	0	0.33

03 Pruning Simulation

- **Sensitivity Analysis**
 - Granularity: Fine-grained
 - Criterion: L1-norm
 - Ratio: 0%~95%까지 5%단위

```
def PSA(model, test_DL, DEVICE):
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0]]
    layers = len(weight_param_list)
    org_acc = Test(model, test_DL, DEVICE)
    results = {}

    pruned_model = copy.deepcopy(model)
    params_dict = dict(pruned_model.named_parameters())

    for i in range(layers):
        target_name, original_param = weight_param_list[i]
        results[target_name] = []
        weight_tensor = original_param.data.clone()

        # criterion
        l1_mag = torch.abs(weight_tensor)
        # granularity
        val, idx = torch.sort(l1_mag.flatten())

        for j in range(20):
            ratio = j / 20.0
            n_prune = int(weight_tensor.numel() * ratio)

            # pruning 마스크 생성
            pruning_mask_flat = torch.ones_like(l1_mag.flatten(), dtype=torch.float32)
            pruning_mask_flat[idx[:n_prune]] = 0
            pruning_mask = pruning_mask_flat.reshape(l1_mag.shape)

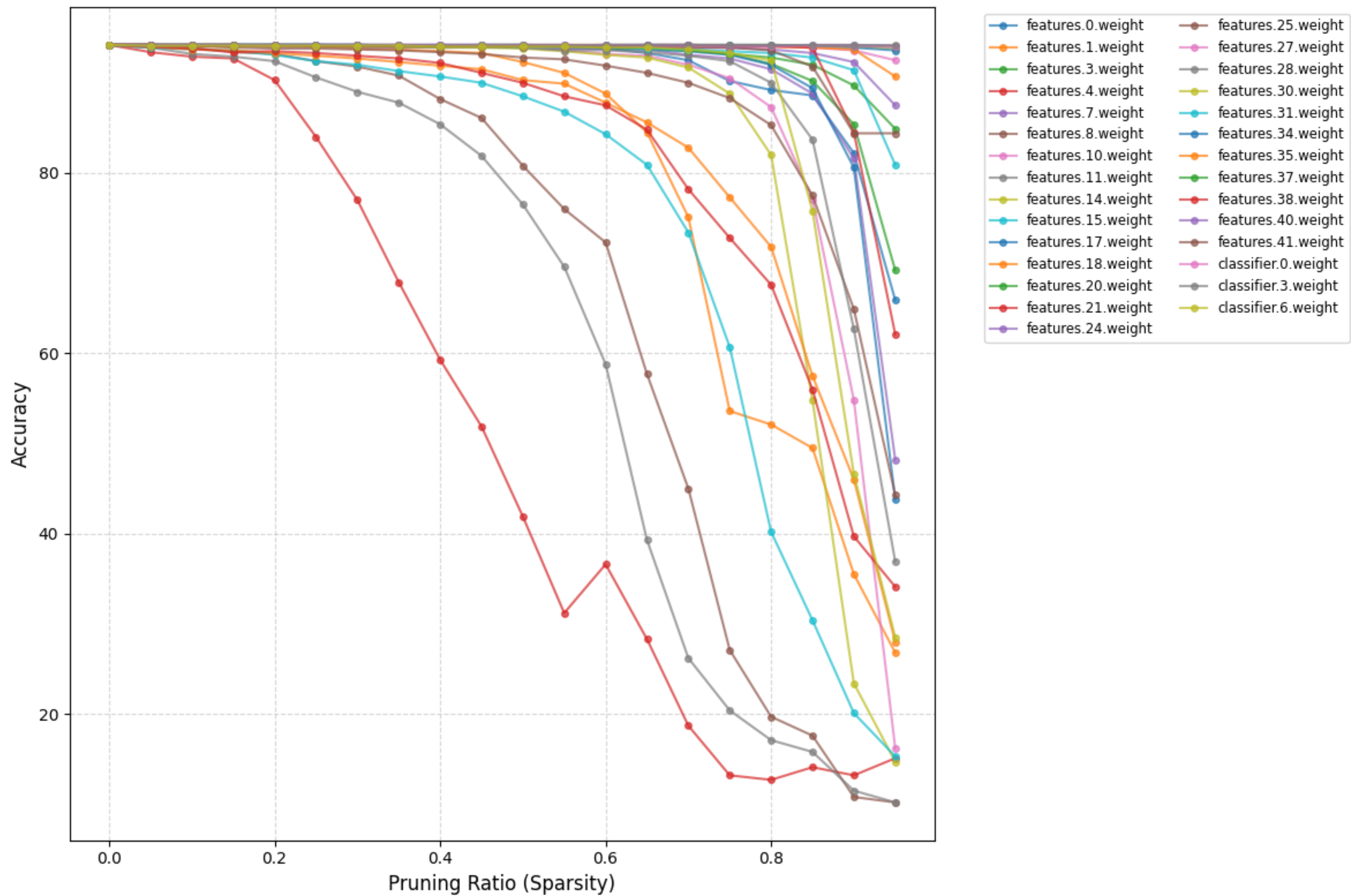
            with torch.no_grad():
                # pruning weight 생성
                pruned_weight = weight_tensor * pruning_mask
                # pruning 적용
                params_dict[target_name].copy_(pruned_weight)

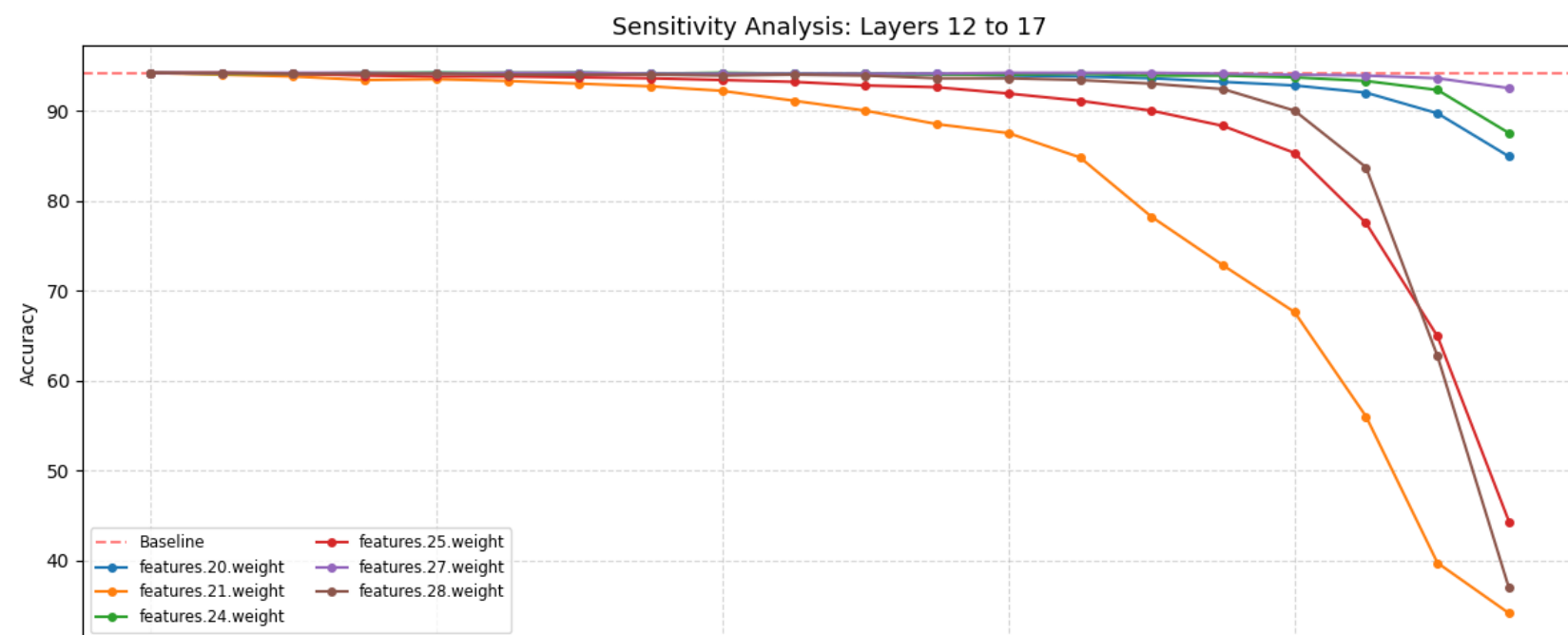
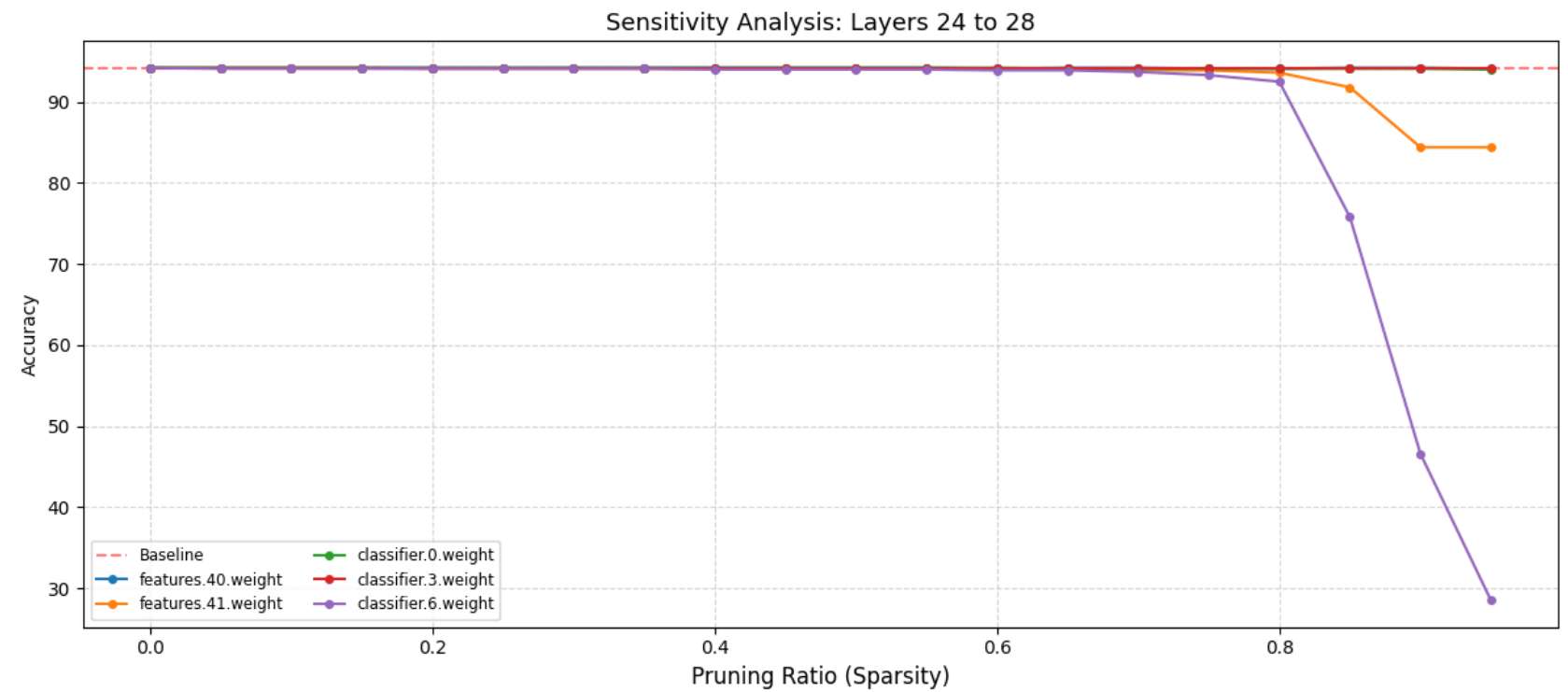
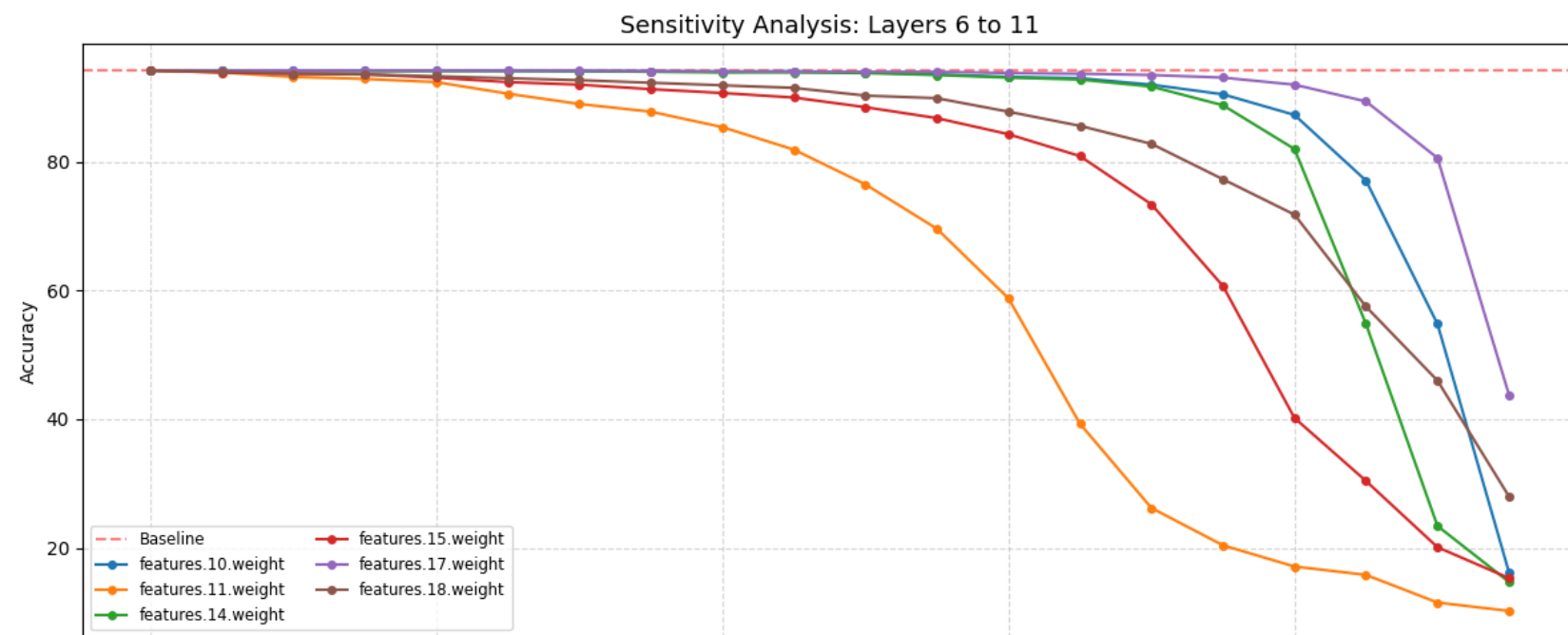
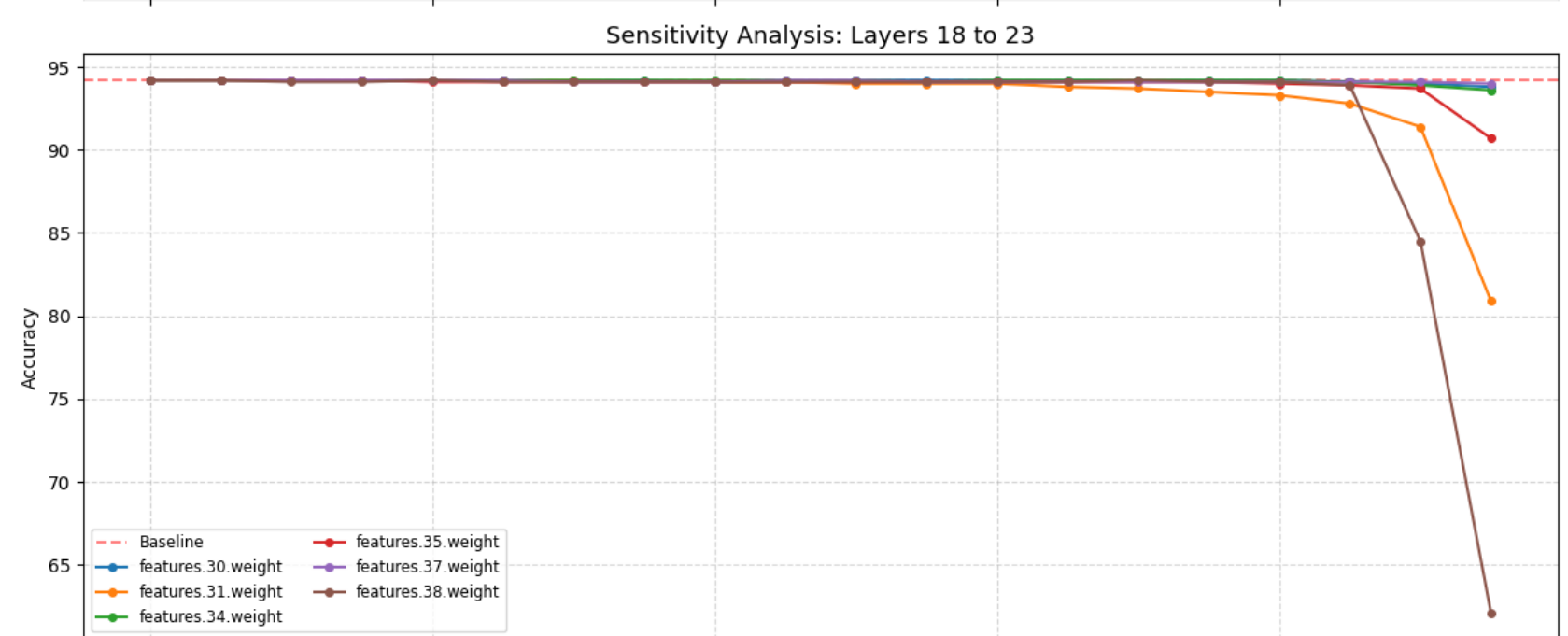
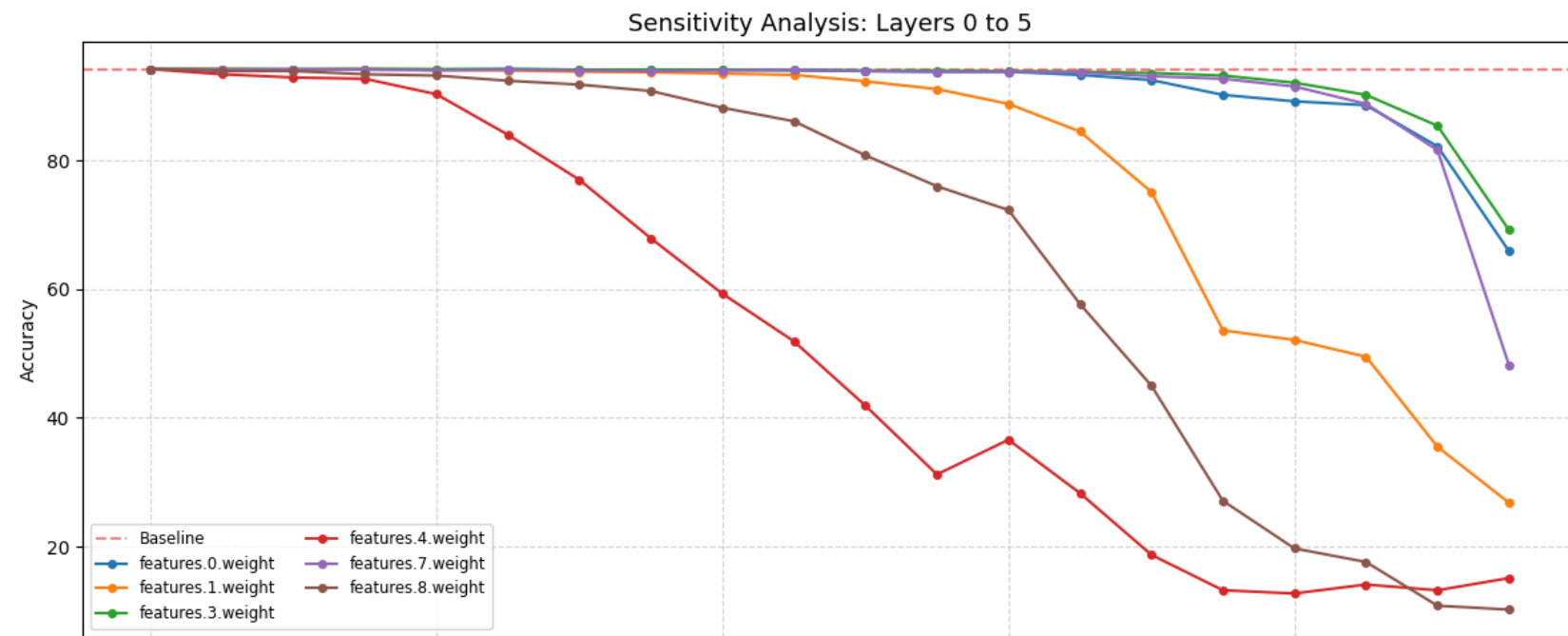
            _, acc = Test(pruned_model, test_DL, DEVICE)
            results[target_name].append({'ratio': ratio, 'acc': acc})

        print(f'Progress: {100*(i*20+j)/(layers*20.0):.2f}%')

    with torch.no_grad():
        # pruning 복구
        params_dict[target_name].copy_(weight_tensor)

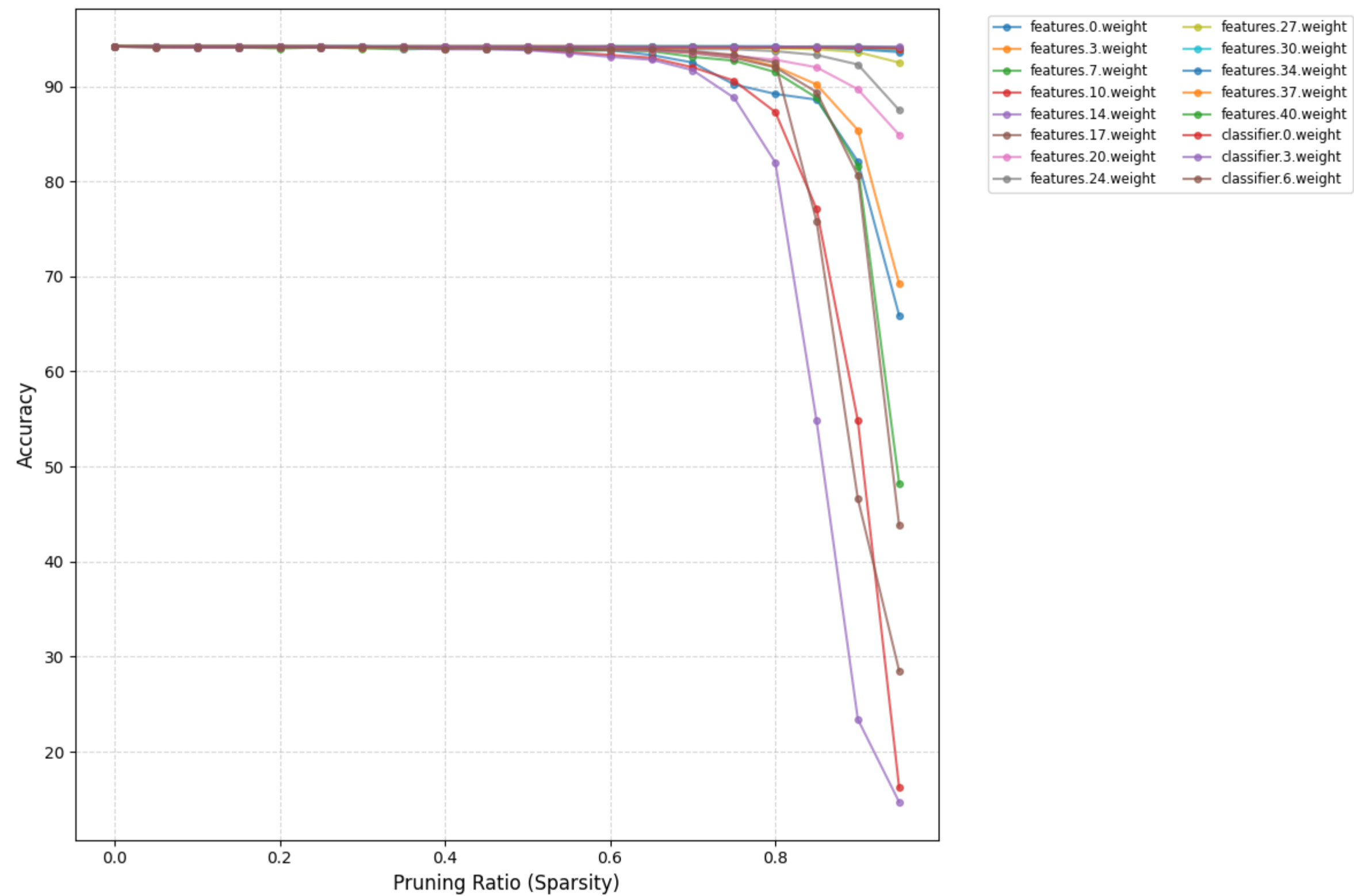
    return results, org_acc
```

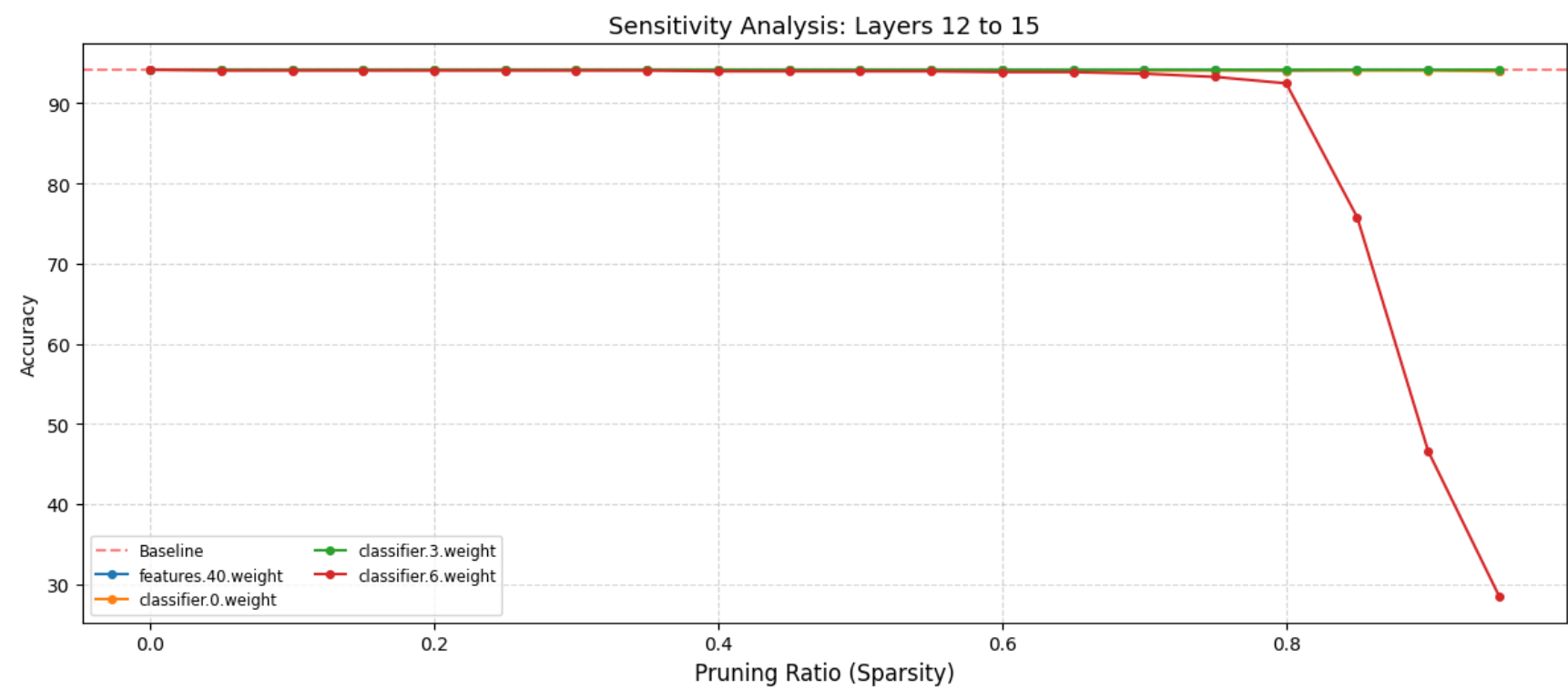
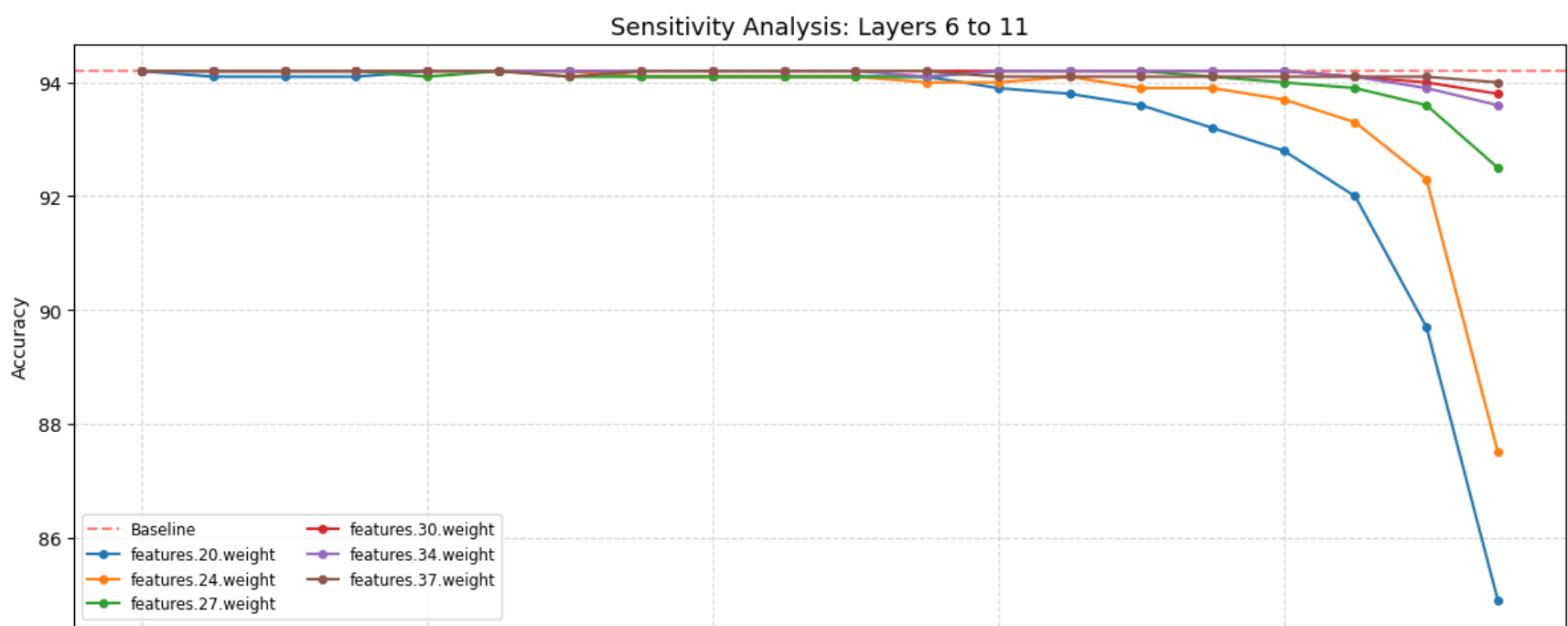
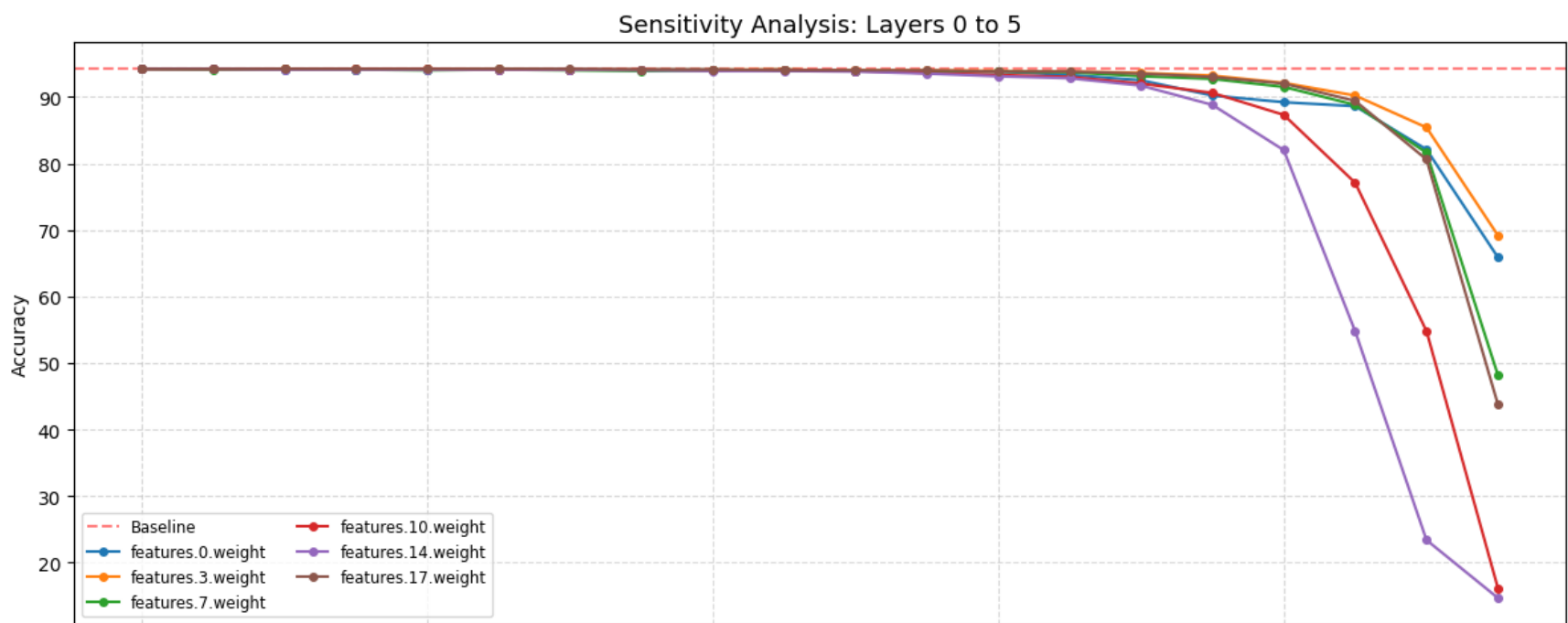




```
def PSA(model, test_DL, DEVICE):  
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0]]  
    layers = len(weight_param_list)  
    org_acc = Test(model, test_DL, DEVICE)  
    results = {}
```

```
def PSA(model, test_DL, DEVICE):  
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() > 1]  
    layers = len(weight_param_list)  
    org_acc = Test(model, test_DL, DEVICE)  
    results = {}
```



03 Pruning Simulation

- **Sensitivity Analysis**
 - Granularity: N:M
 - Criterion: L1-norm
 - Ratio: (2:4), (3:4)

```
def PSA_NM(model, test_DL, DEVICE):
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() > 1]
    layers=len(weight_param_list)
    _, org_acc=Test(model, test_DL, DEVICE)
    results = {}

    pruned_model = copy.deepcopy(model)
    params_dict = dict(pruned_model.named_parameters())
    N,M= 3,4

    for i in range(layers):
        target_name, original_param = weight_param_list[i]
        weight_tensor=original_param.data.clone()
        # print(target_name, original_param.shape)
        # criterion
        l1_mag=torch.abs(weight_tensor)

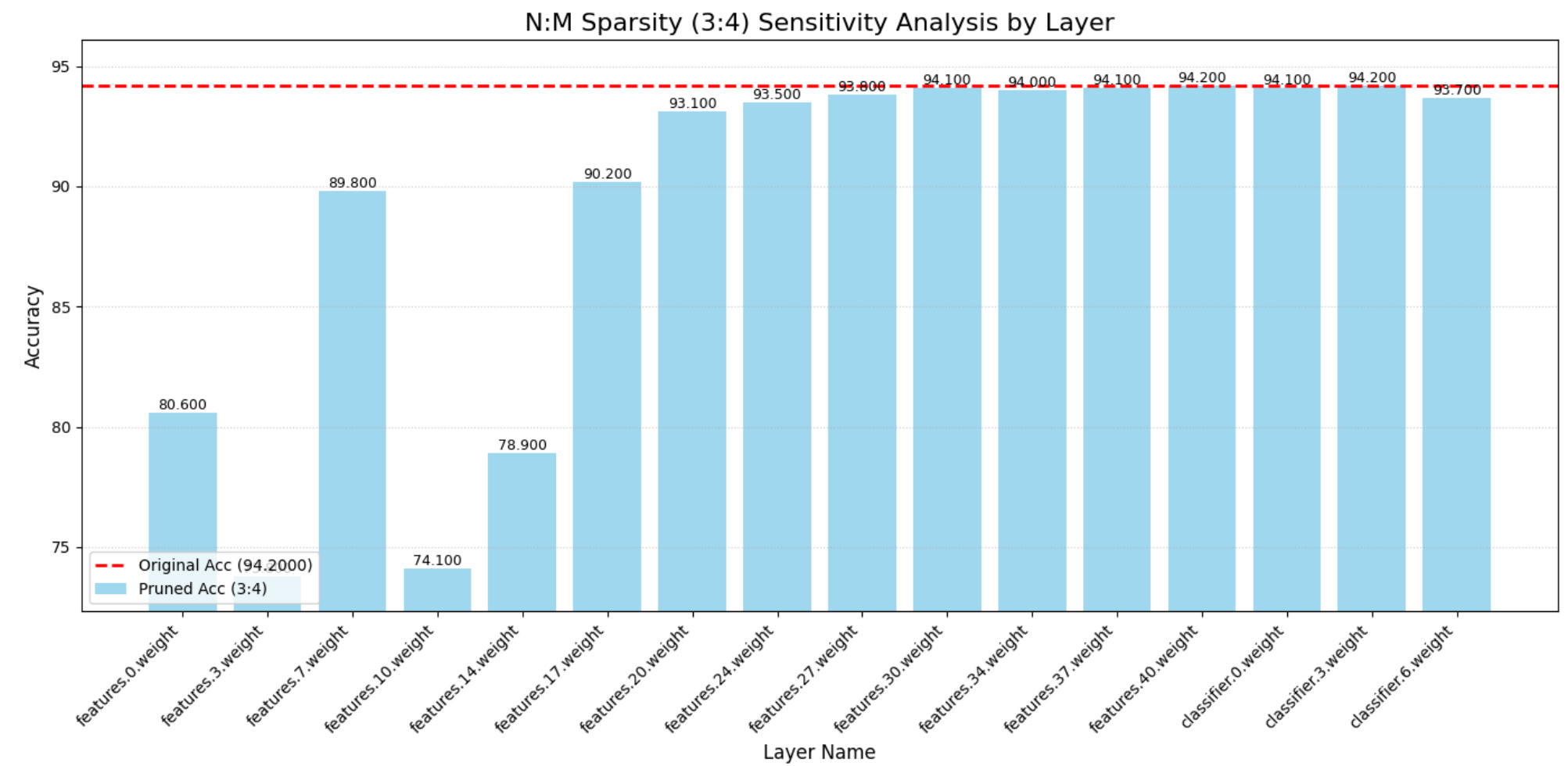
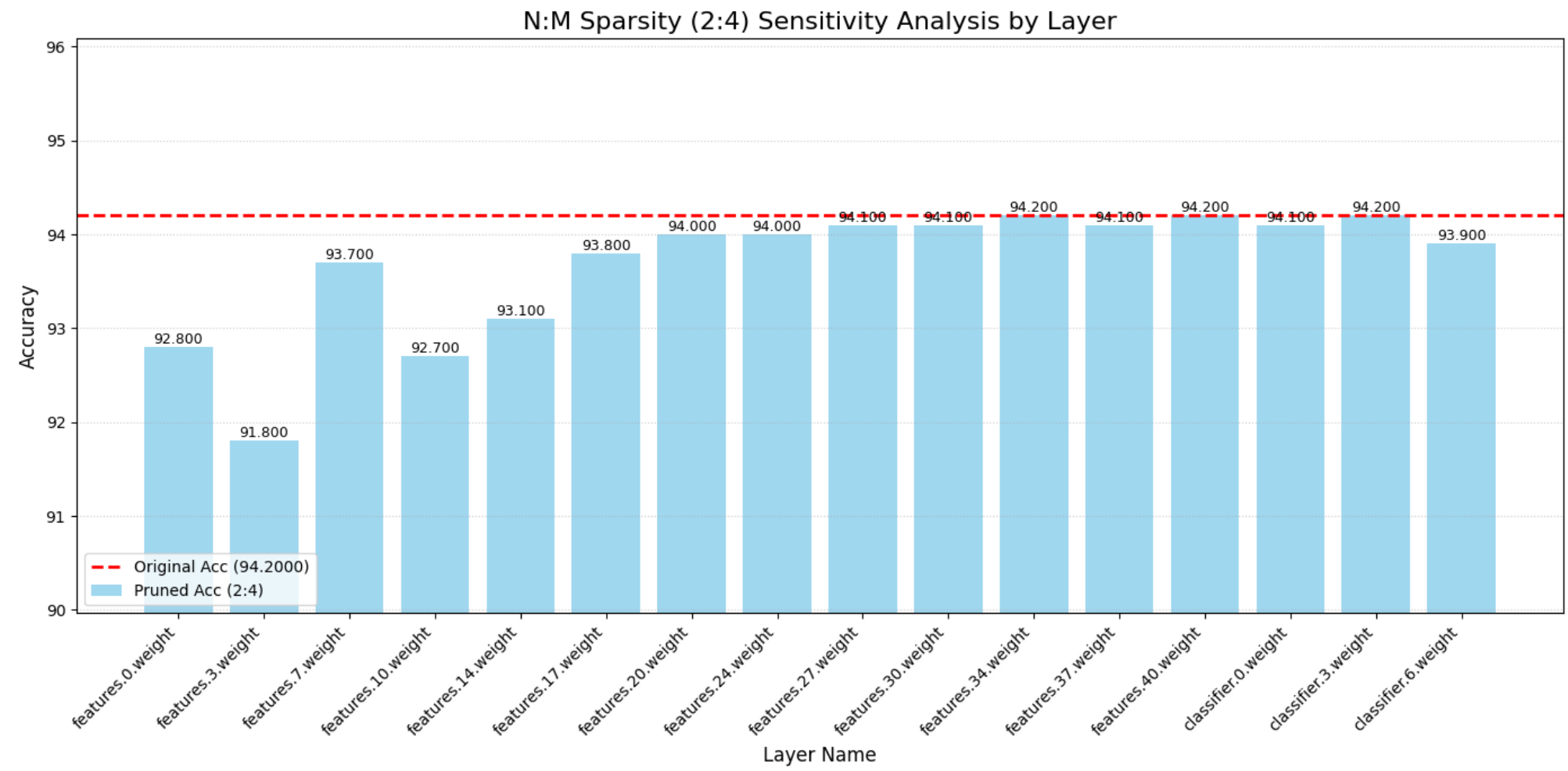
        # granularity
        # print(i)
        # print(target_name)
        val,idx= torch.sort(l1_mag.view(-1,M))
        group=weight_tensor.view(-1,M).shape[0]
        # print(group)
        # print(idx.shape)

        # pruning 마스크 생성
        pruning_mask_flat=torch.ones_like(weight_tensor.view(-1,M), dtype=torch.float32)
        for g in range(group):
            # print(g,idx[g][:N])
            pruning_mask_flat[g][idx[g][:N]]=0
        pruning_mask=pruning_mask_flat.reshape(weight_tensor.shape)

        with torch.no_grad():
            # pruning weight 생성
            pruned_weight=weight_tensor*pruning_mask
            # pruning 적용
            params_dict[target_name].copy_(pruned_weight)

        with torch.no_grad():
            # pruning 복구
            params_dict[target_name].copy_(weight_tensor)

    return results,org_acc
```



03 Pruning Simulation

- **Sensitivity Analysis**

- Granularity: Vector-level pruning
- Criterion: L1-norm
- Ratio: 0%~95%까지 5%단위

```
def PSA_vector(model, test_DL, DEVICE):
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() == 4]
    layers = len(weight_param_list)
    _, org_acc = Test(model, test_DL, DEVICE)
    results = {}

    pruned_model = copy.deepcopy(model)
    params_dict = dict(pruned_model.named_parameters())

    for i in range(layers):
        target_name, original_param = weight_param_list[i]
        results[target_name] = []
        weight_tensor = original_param.data.clone()

        # criterion
        l1_mag = torch.abs(weight_tensor)
        # granularity
        val, idx = torch.sort(torch.sum(l1_mag.view(-1, 3), dim=1))

        # ratio: 0%~95%
        for j in range(20):
            ratio = j / 20.0
            # group = idx.shape[0]
            n_prune = int(idx.shape[0] * ratio)

            # pruning 마스크 생성
            pruning_mask = torch.ones_like(l1_mag.view(-1, 3), dtype=torch.float32)
            pruning_mask[idx[:n_prune]] = 0
            pruning_mask = pruning_mask.reshape(l1_mag.shape)

            with torch.no_grad():
                # pruning weight 생성

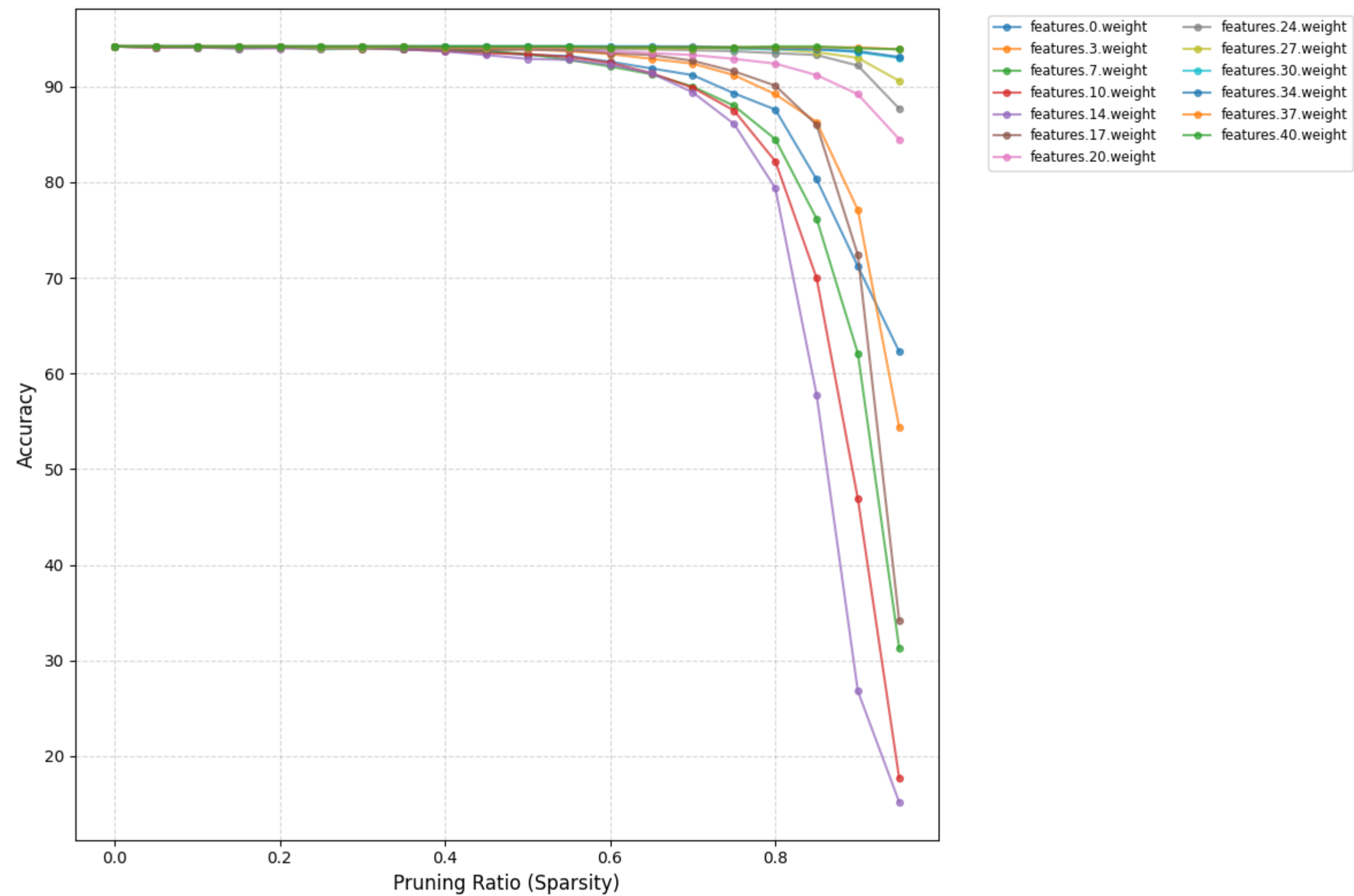
                # print(pruning_mask.shape, weight_tensor.shape)
                pruned_weight = weight_tensor * pruning_mask
                # pruning 적용
                params_dict[target_name].copy_(pruned_weight)

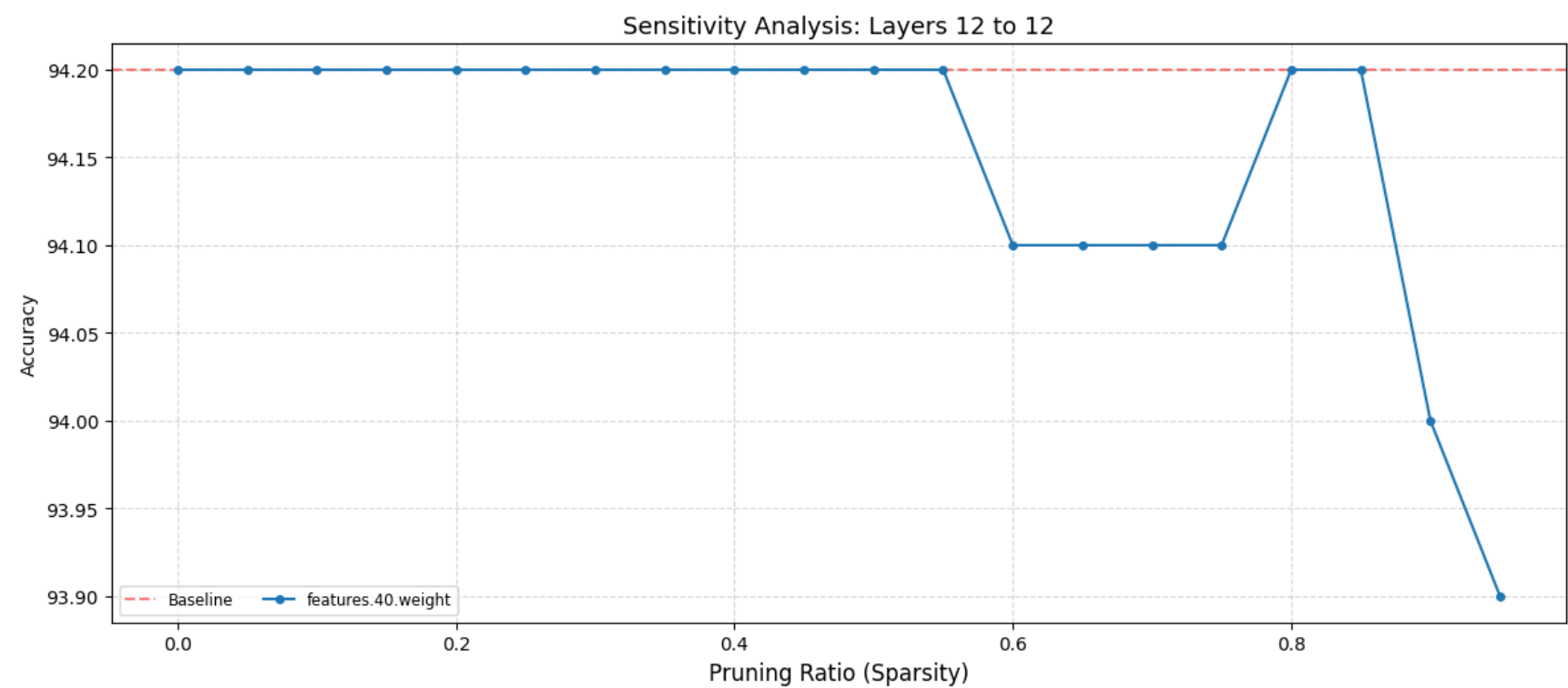
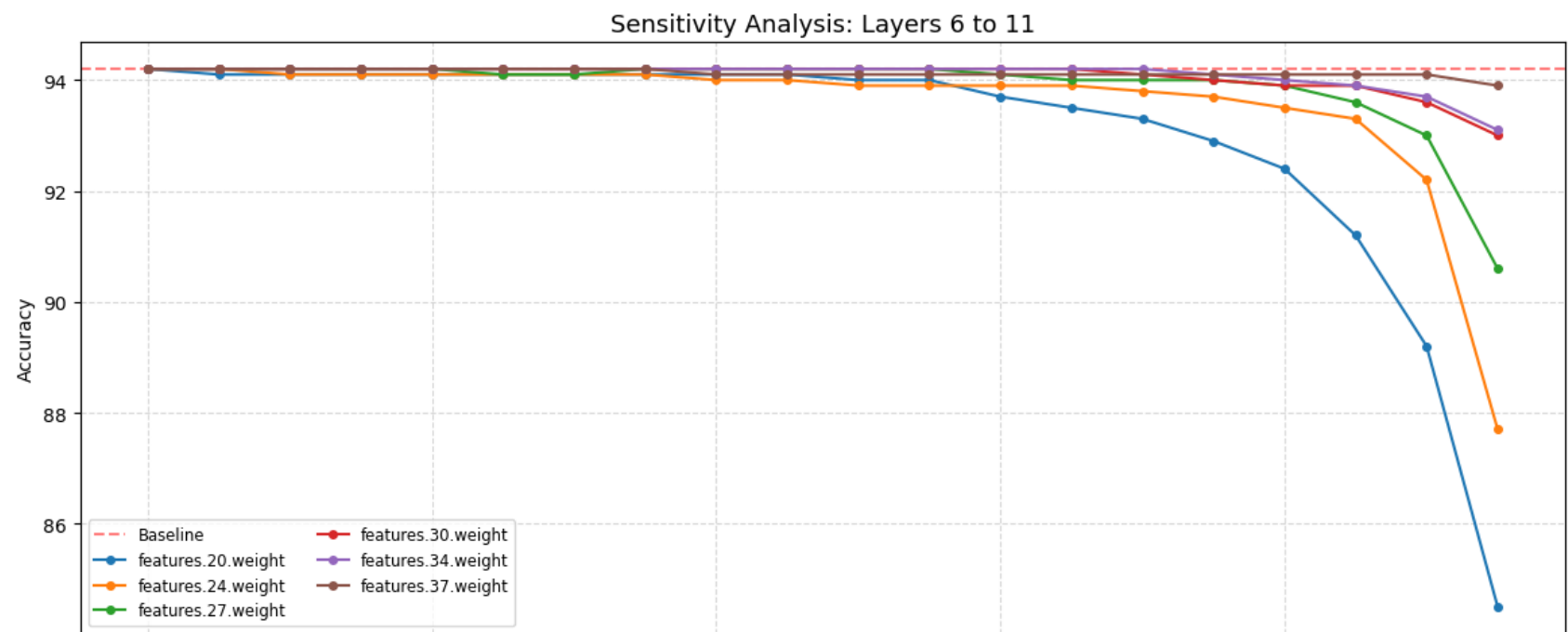
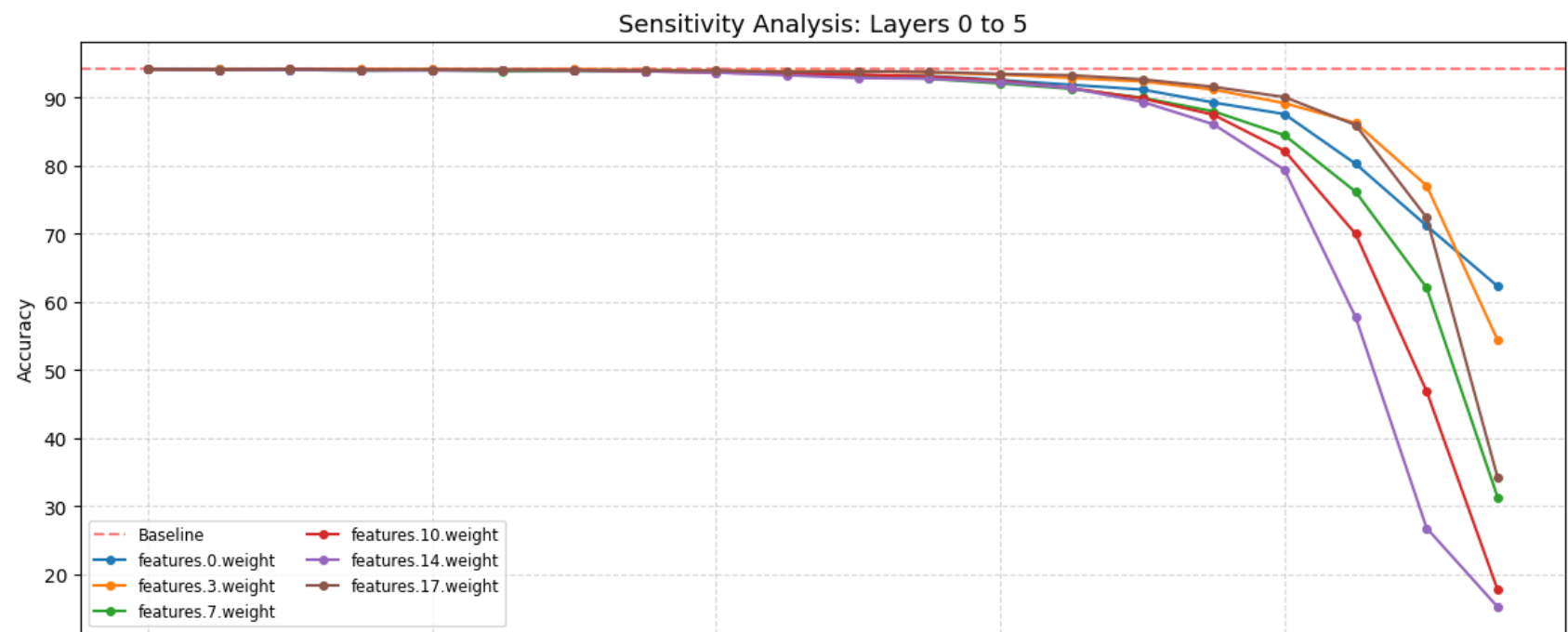
            _, acc = Test(pruned_model, test_DL, DEVICE)
            results[target_name].append({'ratio': ratio, 'acc': acc})

            print(f'Progress: {100*(i*20+j)/(layers*20.0):.2f}%')

        with torch.no_grad():
            # pruning 복구
            params_dict[target_name].copy_(weight_tensor)

    return results, org_acc
```





```

def PSA_kernel(model, test_DL, DEVICE):
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() == 4]
    layers=len(weight_param_list)
    _, org_acc=Test(model, test_DL, DEVICE)
    results = {}

    pruned_model = copy.deepcopy(model)
    params_dict = dict(pruned_model.named_parameters())

    for i in range(layers):
        target_name, original_param = weight_param_list[i]
        results[target_name]=[]
        weight_tensor=original_param.data.clone()

        # criterion
        l1_mag=torch.abs(weight_tensor)
        # granularity
        val,idx= torch.sort(torch.sum(l1_mag.view(-1,9),dim=1))

        # ratio: 0%~95%
        for j in range(20):
            ratio=j/20.0
            # group=idx.shape[0]
            n_prune = int(idx.shape[0] * ratio)

            # pruning 마스크 생성
            pruning_mask=torch.ones_like(l1_mag.view(-1,9), dtype=torch.float32)
            pruning_mask[idx[:n_prune]]=0
            pruning_mask=pruning_mask.reshape(l1_mag.shape)

            with torch.no_grad():
                # pruning weight 생성

                # print(pruning_mask.shape, weight_tensor.shape)
                pruned_weight=weight_tensor*pruning_mask
                # pruning 적용
                params_dict[target_name].copy_(pruned_weight)

            _, acc = Test(pruned_model, test_DL, DEVICE)
            results[target_name].append({'ratio': ratio, 'acc': acc})

            print(f'Progress: {100*(i*20+j)/(layers*20.0):.2f}%')

        with torch.no_grad():
            # pruning 복구
            params_dict[target_name].copy_(weight_tensor)

    return results,org_acc

```

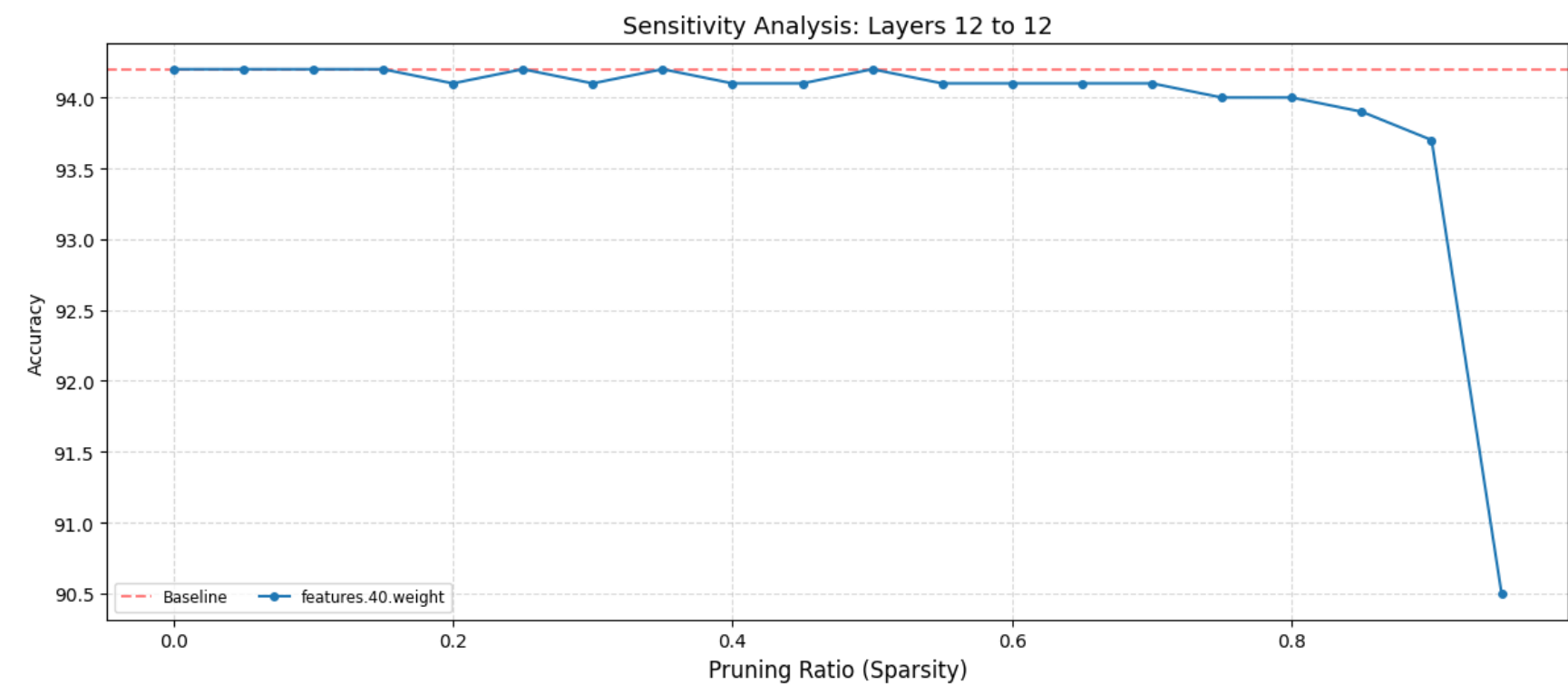
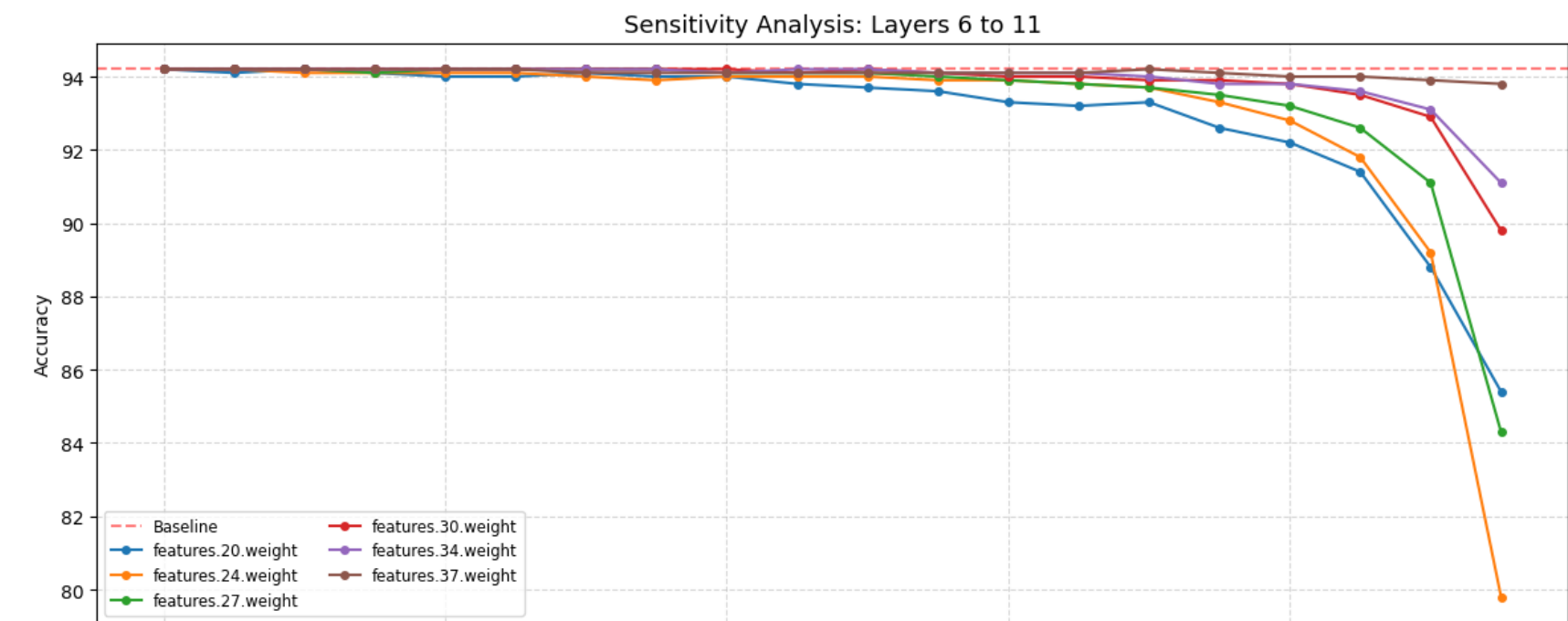
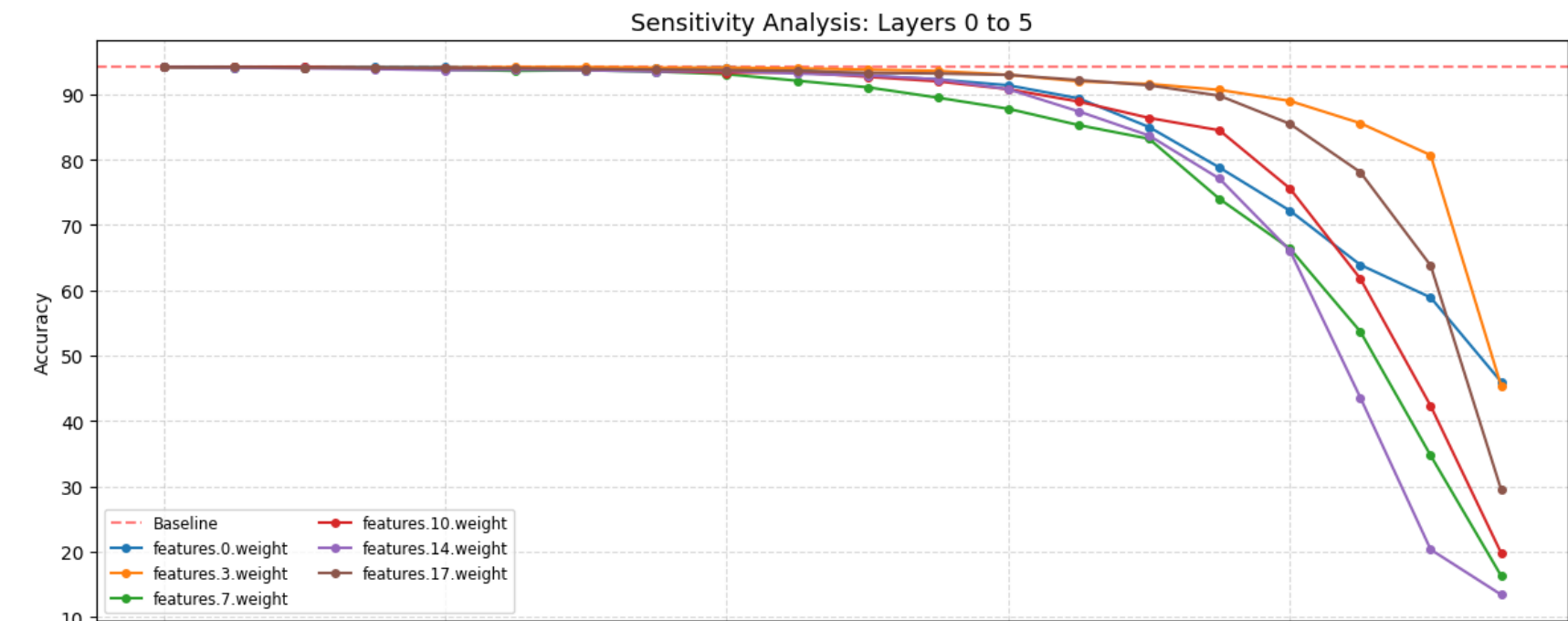
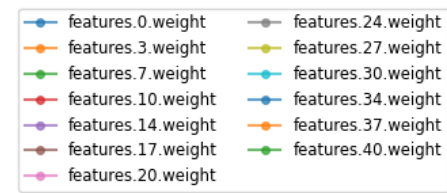
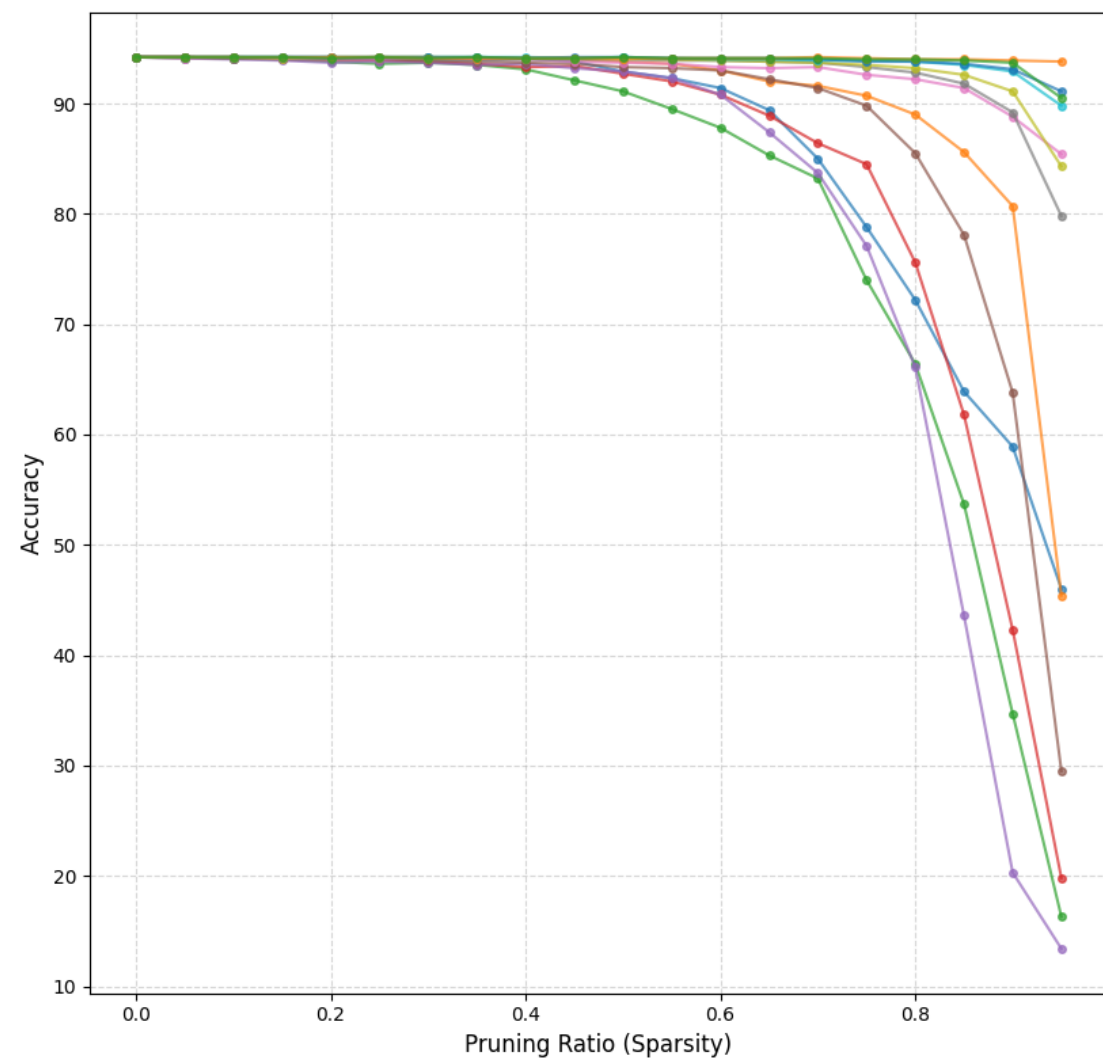
```

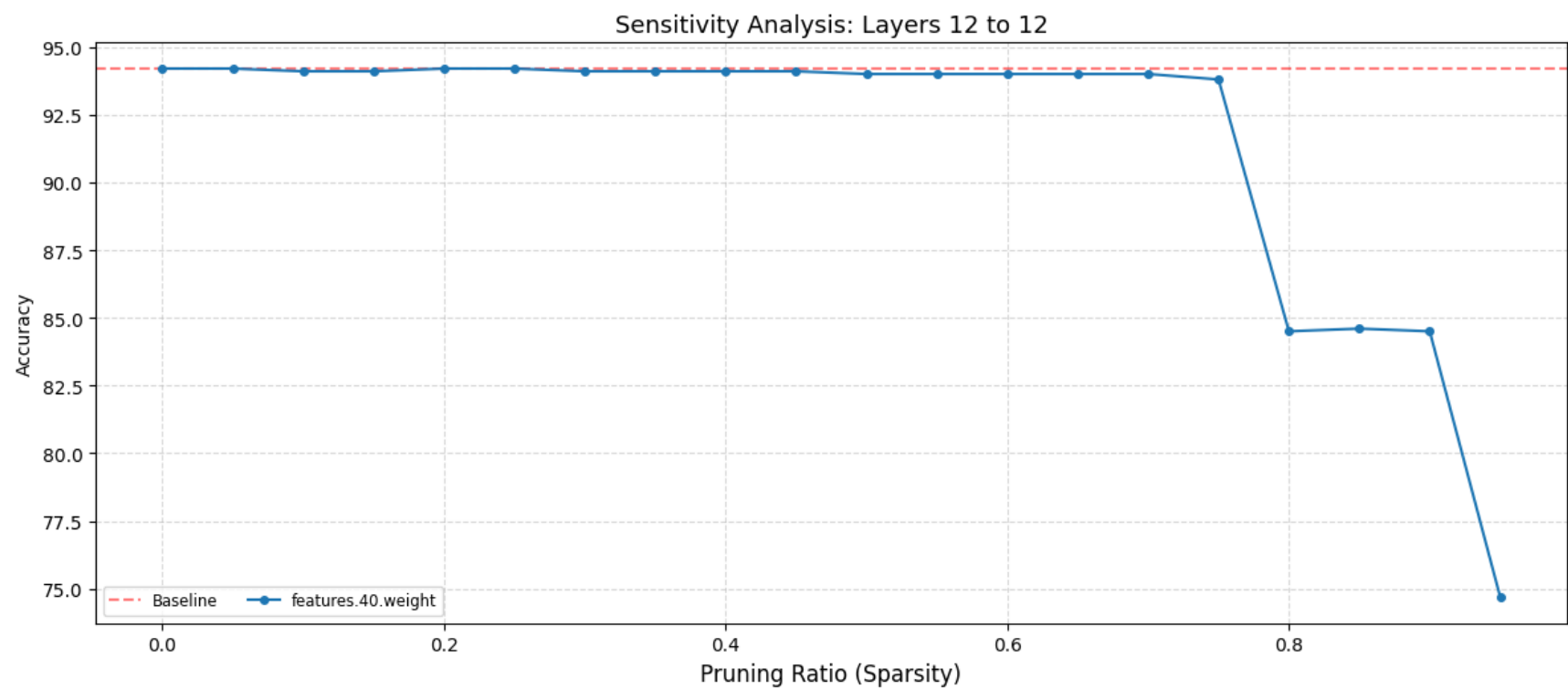
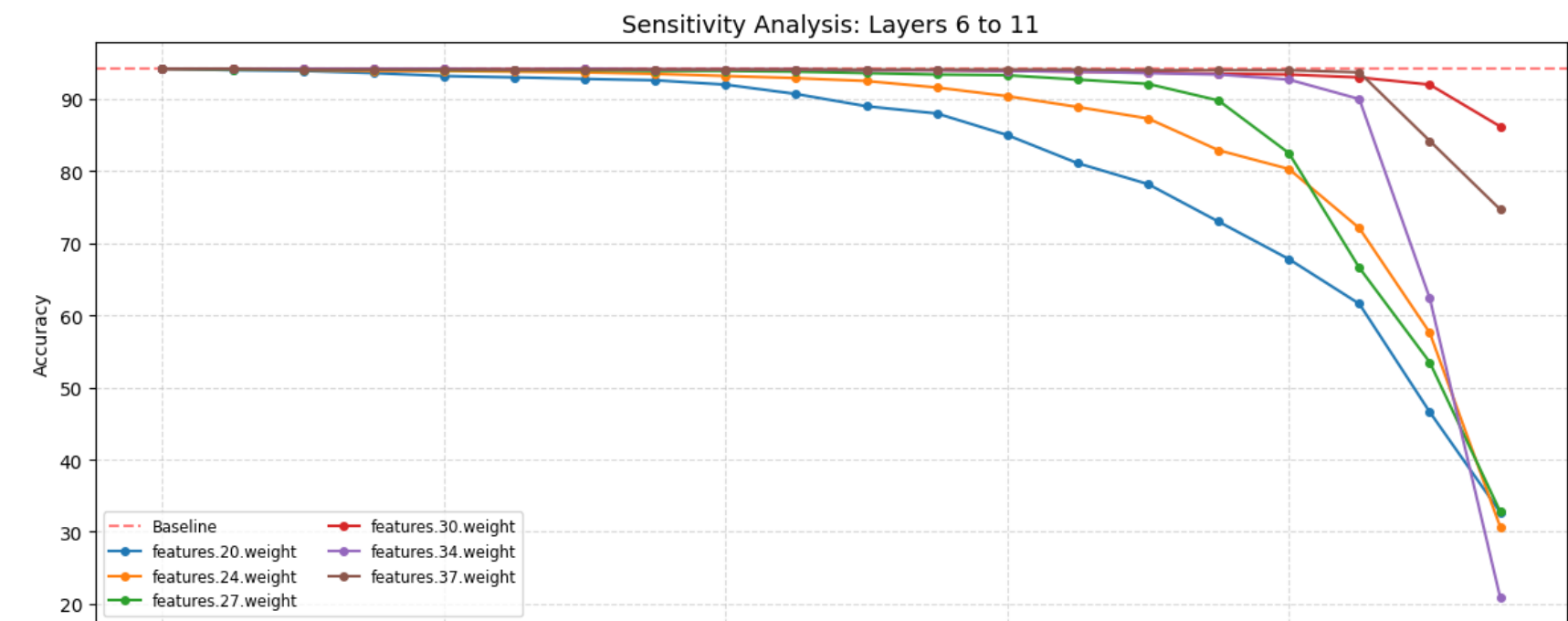
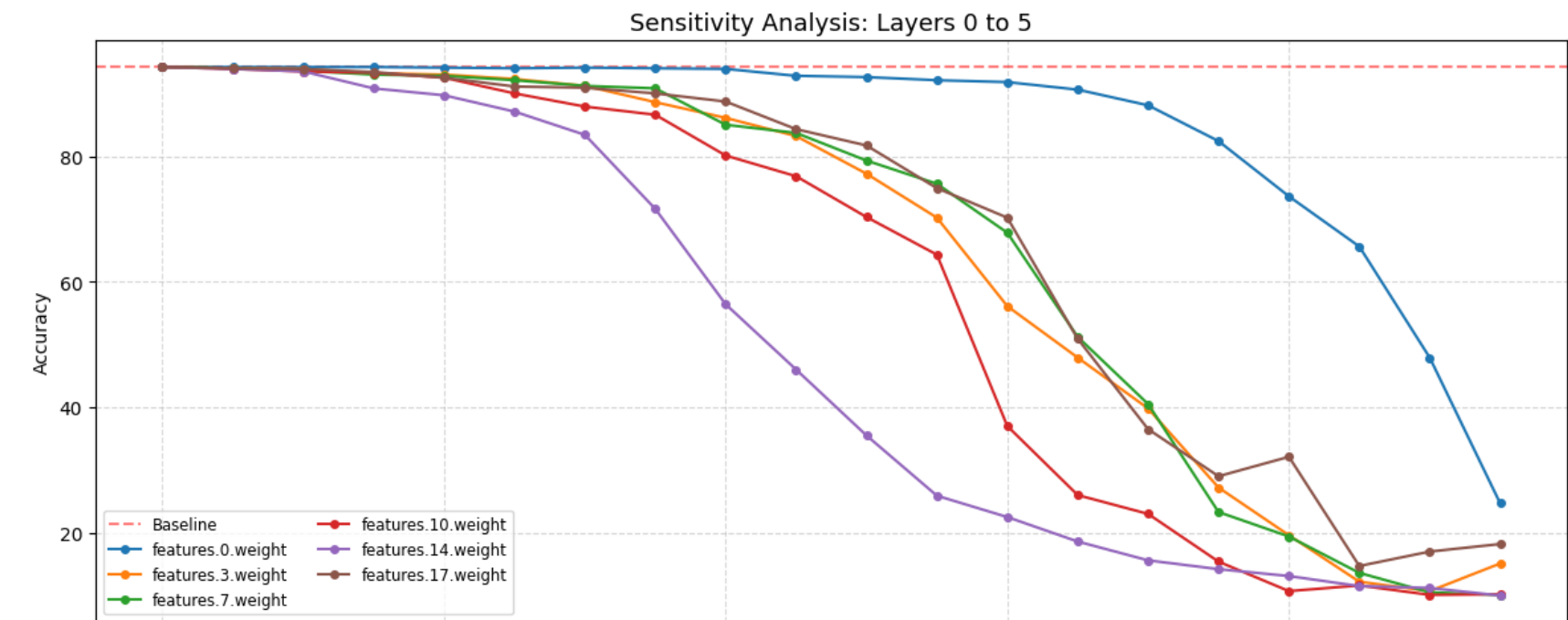
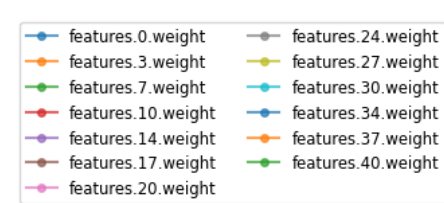
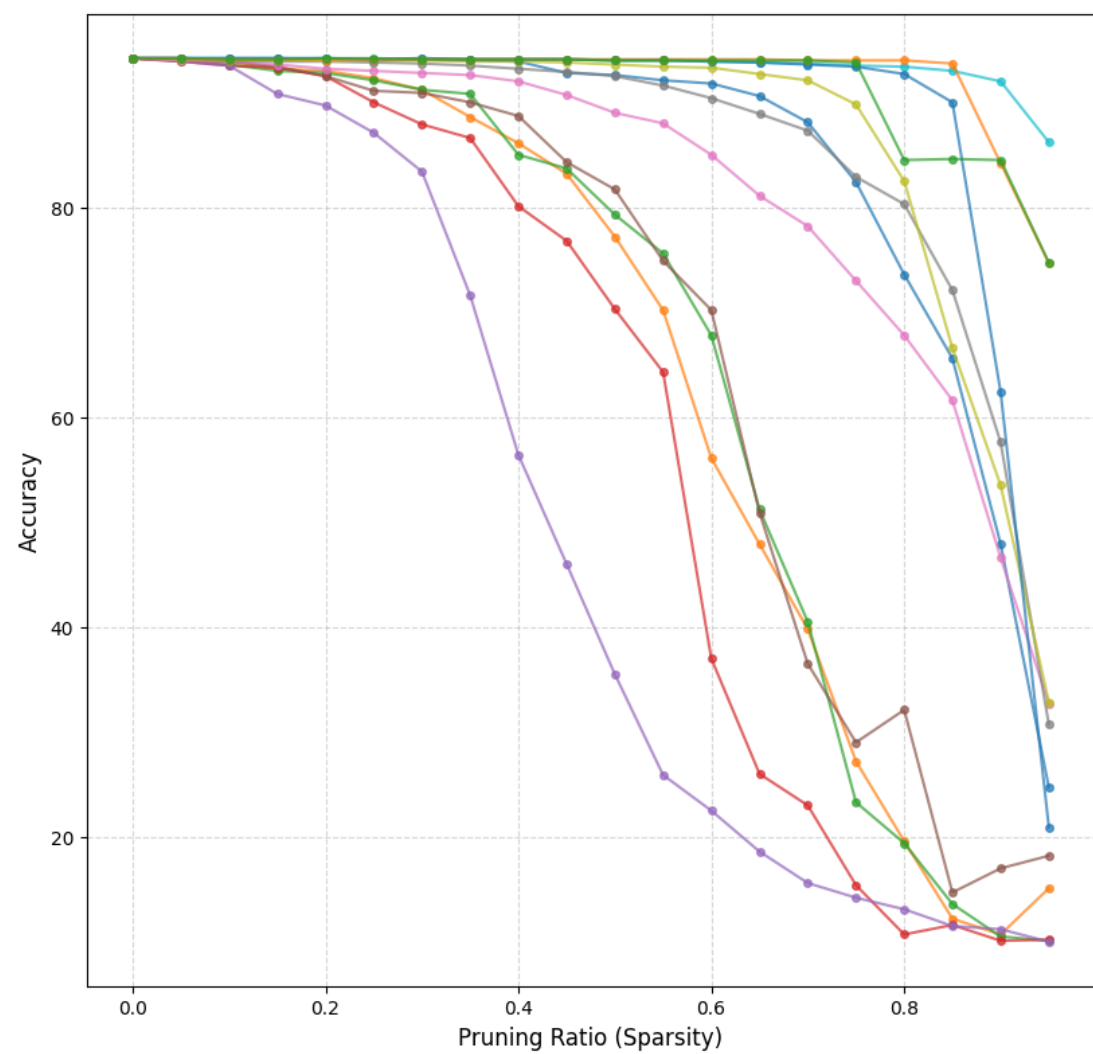
# criterion
l1_mag=torch.abs(weight_tensor)
# granularity
val,idx= torch.sort(torch.sum(l1_mag.view(l1_mag.shape[0],l1_mag.shape[1]*l1_mag.shape[2]*l1_mag.shape[3]),dim=1))

# ratio: 0%~95%
for j in range(20):
    ratio=j/20.0
    # group=idx.shape[0]
    n_prune = int(idx.shape[0] * ratio)

    # pruning 마스크 생성
    pruning_mask=torch.ones_like(l1_mag.view(l1_mag.shape[0],l1_mag.shape[1]*l1_mag.shape[2]*l1_mag.shape[3]), dtype=torch.float32)
    pruning_mask[idx[:n_prune]]=0
    pruning_mask=pruning_mask.reshape(l1_mag.shape)

```



03 Pruning Simulation

- **Sensitivity Analysis**
 - Granularity: Channel-level pruning
 - Criterion: scaling ratio
 - Ratio: 0%~95%까지 5%단위

```
def PSA_scaling(model, test_DL, DEVICE):
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() == 1]
    layers = len(weight_param_list)
    _, org_acc = Test(model, test_DL, DEVICE)
    results = {}

    pruned_model = copy.deepcopy(model)
    params_dict = dict([pruned_model.named_parameters()])

    pbar = tqdm(total=layers * 20, desc="Sensitivity Analysis")
    for i in range(layers):
        target_name, original_param = weight_param_list[i]
        results[target_name] = []
        weight_tensor = original_param.data.clone()

        # criterion
        l1_mag = torch.abs(weight_tensor)
        # granularity
        val, idx = torch.sort(l1_mag, dim=0)

        # ratio: 0%~95%
        for j in range(20):
            ratio = j / 20.0
            # group = idx.shape[0]
            n_prune = int(idx.shape[0] * ratio)

            # pruning 마스크 생성
            pruning_mask = torch.ones_like(l1_mag, dtype=torch.float32)
            pruning_mask[idx[:n_prune]] = 0
            pruning_mask = pruning_mask.reshape(l1_mag.shape)

            with torch.no_grad():
                # pruning weight 생성

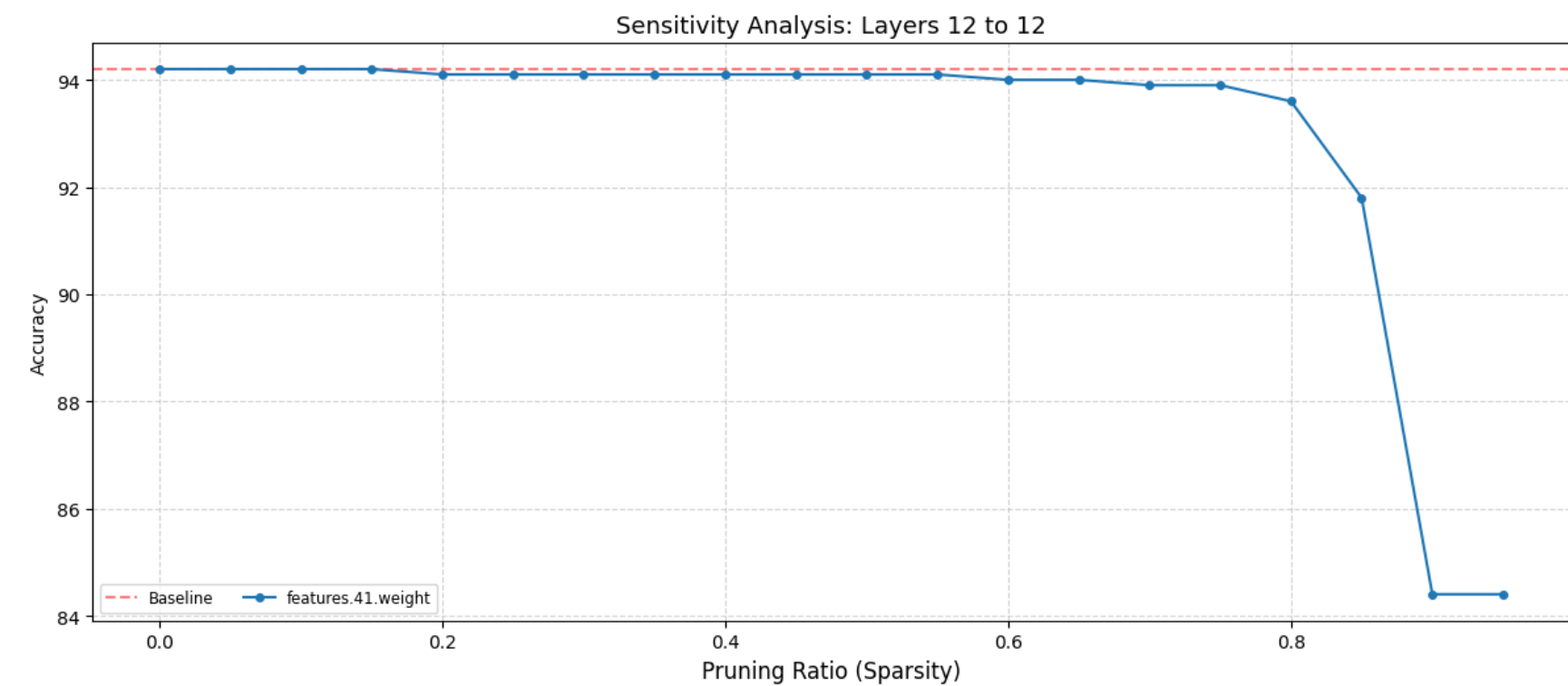
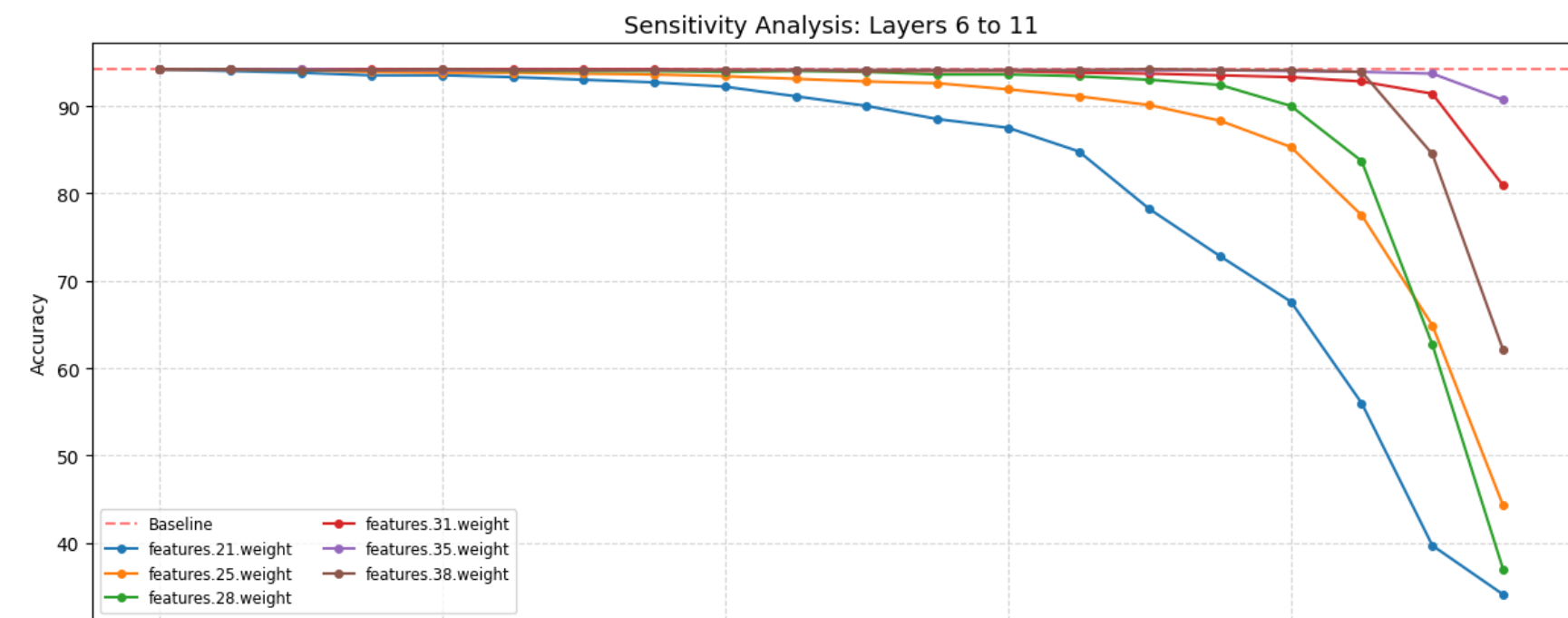
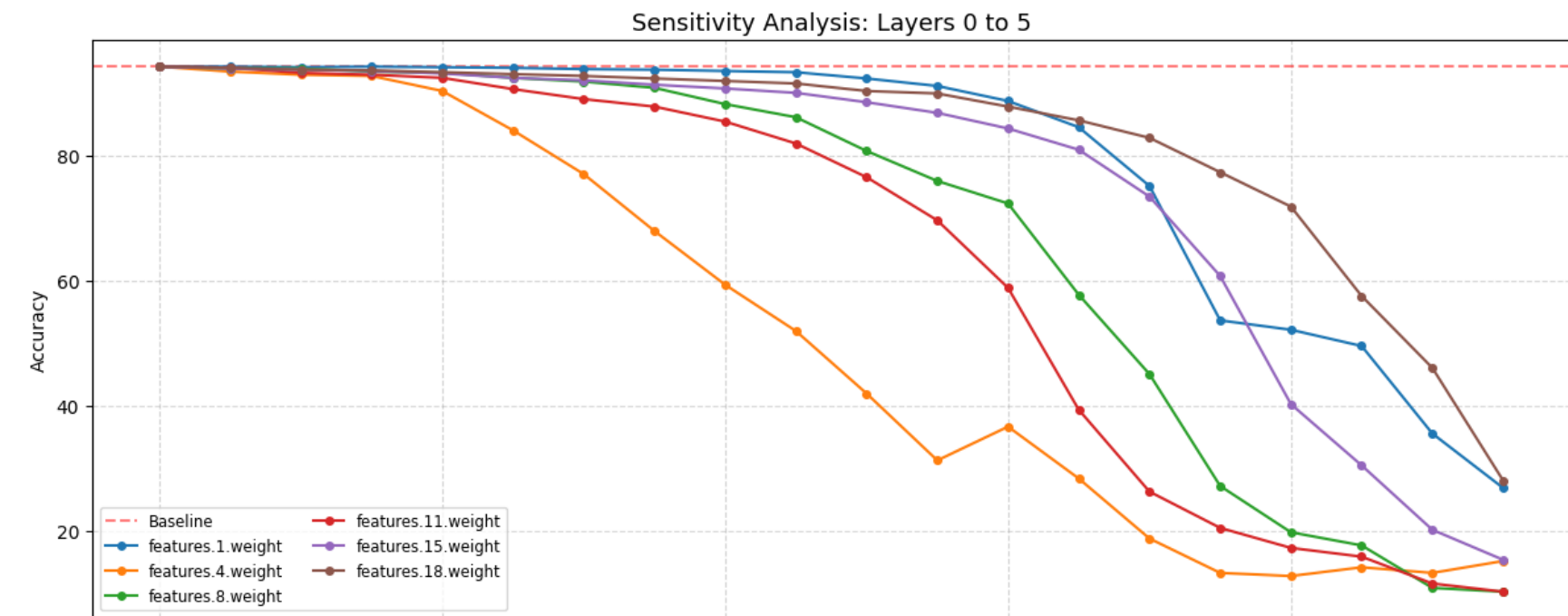
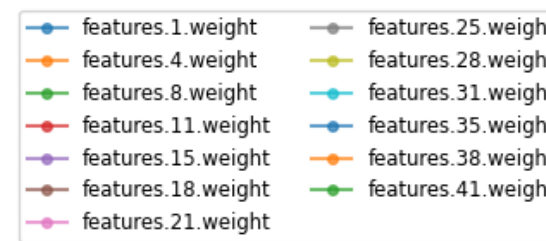
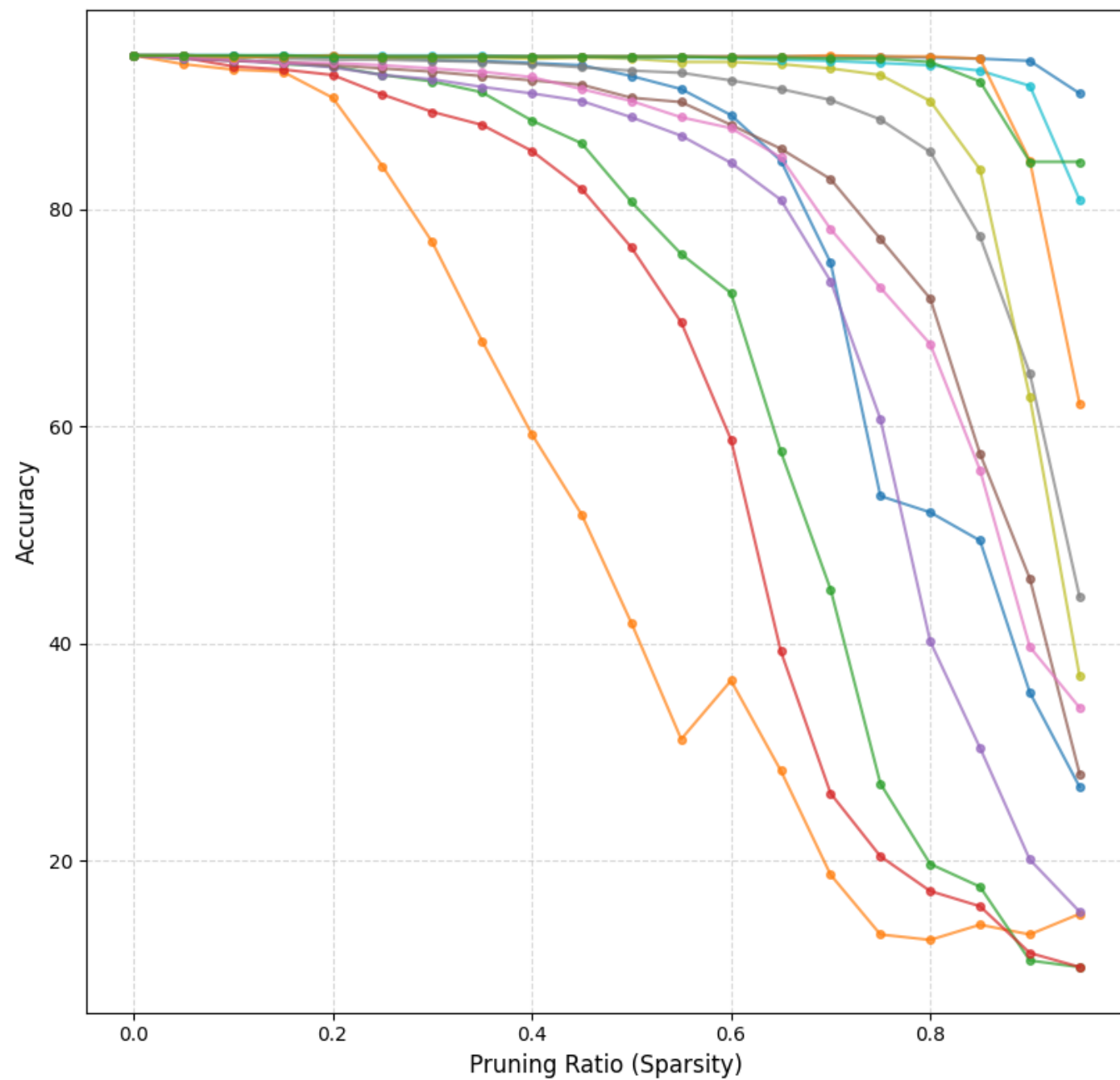
                # print(pruning_mask.shape, weight_tensor.shape)
                pruned_weight = weight_tensor * pruning_mask
                # pruning 적용

            pbar.set_description(f"Layer {i+1}/{layers} [{target_name}]")
            _, acc = Test(pruned_model, test_DL, DEVICE)
            results[target_name].append({'ratio': ratio, 'acc': acc})

            # print(f'Progress: {100*(i*20+j)/(layers*20.0):.2f}%', end='\r', flush=True)
            pbar.update(1)
            pbar.set_postfix(ratio=f"{ratio:.2f}", acc=f"{acc:.2f}")

        with torch.no_grad():
            # pruning 복구
            params_dict[target_name].copy_(weight_tensor)

    pbar.close()
    return results, org_acc
```



03 Pruning Simulation –Summary

Acc_threshold	Pruning 후 accuracy	Fine-tuning 후 accuracy
1%	1170/10000 (11.7 %)	9369/10000 (93.7 %)
0.5%	3289/10000 (32.9 %)	9378/10000 (93.8 %)
0.25%	7998/10000 (80.0 %)	9408/10000 (94.1 %)
0.1%	9416/10000 (94.2 %)	9415/10000 (94.2 %)
2%	1000/10000 (10.0 %)	9332/10000 (93.3 %)
5%	1000/10000 (10.0 %)	9239/10000 (92.4 %)
10%	1000/10000 (10.0 %)	9171/10000 (91.7 %)

- Sensitivity analysis에서 각 layer의 accuracy drop이 Acc_threshold 가 되도록 하는 pruning rate를 구함
- 각 layer를 해당 ratio로 pruning을 한 후 accuracy 측정
- Pruning 하면서 생기는 pruning mask를 이용하여 3-epoch fine-tuning 하여 accuracy 복구 확인

Threshold Drop: 0.10%
Test accuracy: 9416/10000 (94.2 %)
Epoch [1/3] | Loss: 0.0510 | Test Acc: 94.10%
Epoch [2/3] | Loss: 0.0233 | Test Acc: 94.10%
Epoch [3/3] | Loss: 0.0148 | Test Acc: 94.20%
Test accuracy: 9415/10000 (94.2 %)

Threshold Drop: 0.25%
Test accuracy: 7998/10000 (80.0 %)
Epoch [1/3] | Loss: 0.3124 | Test Acc: 93.90%
Epoch [2/3] | Loss: 0.0636 | Test Acc: 94.10%
Epoch [3/3] | Loss: 0.0330 | Test Acc: 94.10%
Test accuracy: 9408/10000 (94.1 %)

Threshold Drop: 0.50%
Test accuracy: 3289/10000 (32.9 %)
Epoch [1/3] | Loss: 0.4682 | Test Acc: 93.70%
Epoch [2/3] | Loss: 0.0895 | Test Acc: 93.90%
Epoch [3/3] | Loss: 0.0433 | Test Acc: 93.80%
Test accuracy: 9378/10000 (93.8 %)

Threshold Drop: 2.00%
Test accuracy: 1000/10000 (10.0 %)
Epoch [1/3] | Loss: 0.7414 | Test Acc: 93.00%
Epoch [2/3] | Loss: 0.1574 | Test Acc: 93.10%
Epoch [3/3] | Loss: 0.0722 | Test Acc: 93.30%
Test accuracy: 9332/10000 (93.3 %)

Threshold Drop: 5.00%
Test accuracy: 1000/10000 (10.0 %)
Epoch [1/3] | Loss: 0.8128 | Test Acc: 91.60%
Epoch [2/3] | Loss: 0.1977 | Test Acc: 92.40%
Epoch [3/3] | Loss: 0.0959 | Test Acc: 92.40%
Test accuracy: 9239/10000 (92.4 %)

Threshold Drop: 10.00%
Test accuracy: 1000/10000 (10.0 %)
Epoch [1/3] | Loss: 0.9084 | Test Acc: 90.50%
Epoch [2/3] | Loss: 0.2534 | Test Acc: 91.60%
Epoch [3/3] | Loss: 0.1385 | Test Acc: 91.70%
Test accuracy: 9171/10000 (91.7 %)

features.0.weight	0.85	88.60%
features.3.weight	0.90	85.40%
features.7.weight	0.85	88.80%
features.10.weight	0.80	87.30%
features.14.weight	0.75	88.80%
features.17.weight	0.85	89.40%
features.20.weight	0.95	84.90%
features.24.weight	0.95	87.50%
features.27.weight	0.95	92.50%
features.30.weight	0.95	93.80%
features.34.weight	0.95	93.60%
features.37.weight	0.95	94.00%
features.40.weight	0.95	94.10%
classifier.0.weight	0.95	94.00%
classifier.3.weight	0.95	94.20%
classifier.6.weight	0.80	92.50%

■ CSR format?

1	7		
5		3	9
	2	8	
			6



Row pointer

Column
Index

Value

0	2	5	7	8			
0	1	0	2	3	1	2	3
1	7	5	3	9	2	8	6

• 32x32행렬의 Dense Format 용량

- 32bit X 64

• 32x32행렬의 CSR Format 용량

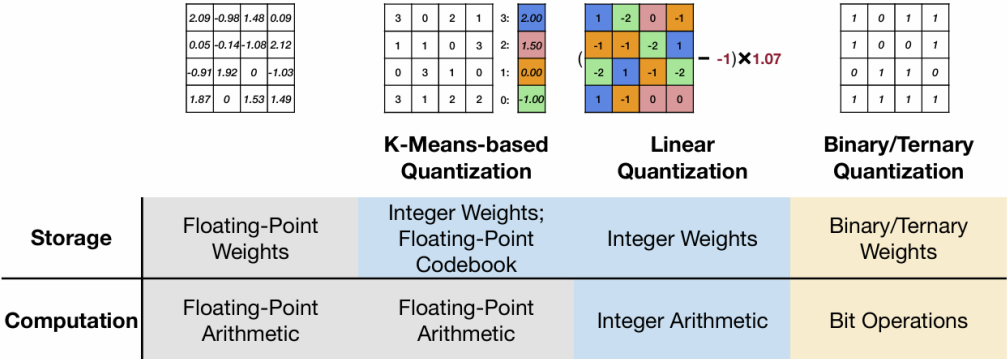
- Value 메모리: 32bit X NNZ
- Column Index 메모리: 5bit X NNZ
- Row pointer 메모리: 32 X (최대 32x32개 =10비트)

04 About Quantization

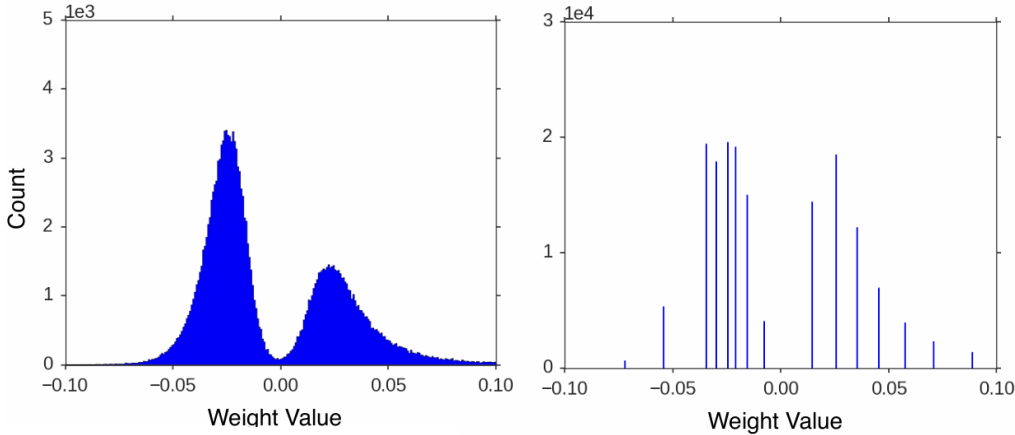


■

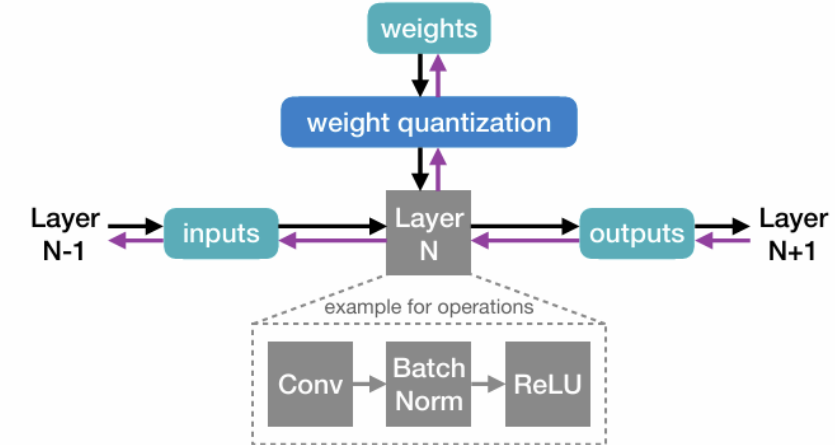
Quantization method



Post-Training Quantization



Quantization Aware Training



04 About Quantization – Method

• FP weights

[IEEE 754](#) Single Precision 32-bit Float (IEEE FP32)



[IEEE 754](#) Half Precision 16-bit Float (IEEE FP16)



[Google](#) Brain Float (BF16)



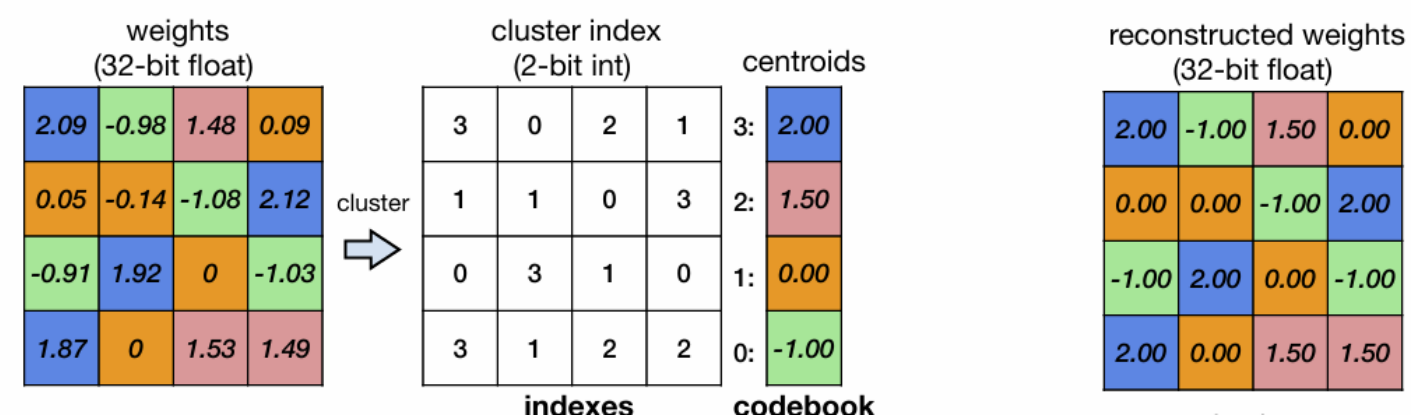
INT4			
S			
0	0	0	1
0	1	1	1

FP4 (E1M2)			
S	E	M	M
0	0	0	1
0	1	1	1

FP4 (E2M1)			
S	E	E	M
0	0	0	1
0	1	1	1

FP4 (E3M0)			
S	E	E	E
0	0	0	1
0	1	1	1

• K-means based quantization

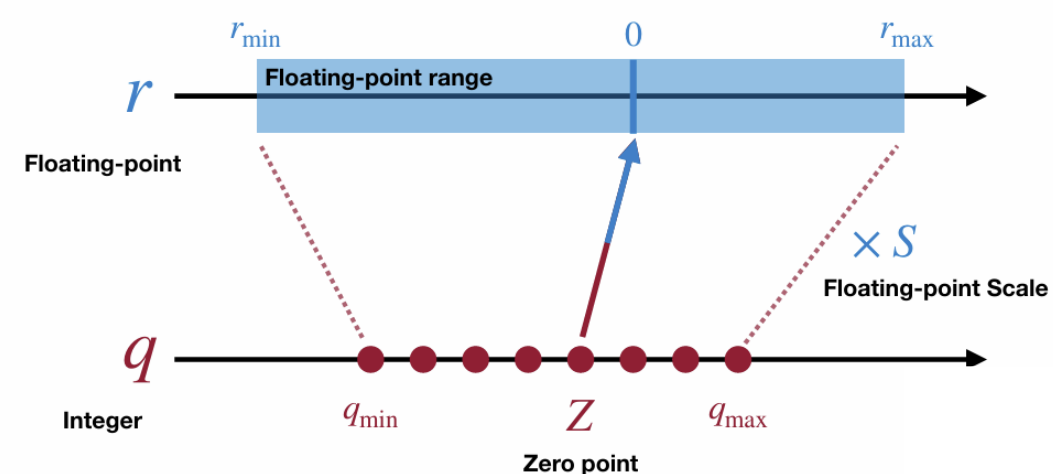


- Data 표현 방식 자체를 바꾸는 방식
- 별도의 값 필요하지 않음
- 데이터의 형태와 계산 방식 모두 FP/INT

- 데이터를 n개로 clustering
- Cluster index와 Codebook을 저장
- Cluster가 n개면 index에 $\log_2(n)$ 개의 비트 필요
예) 32개 cluster → 5비트 필요
- 데이터의 형태: index는 INT, codebook은 FP
- 연산 방식: FP 연산

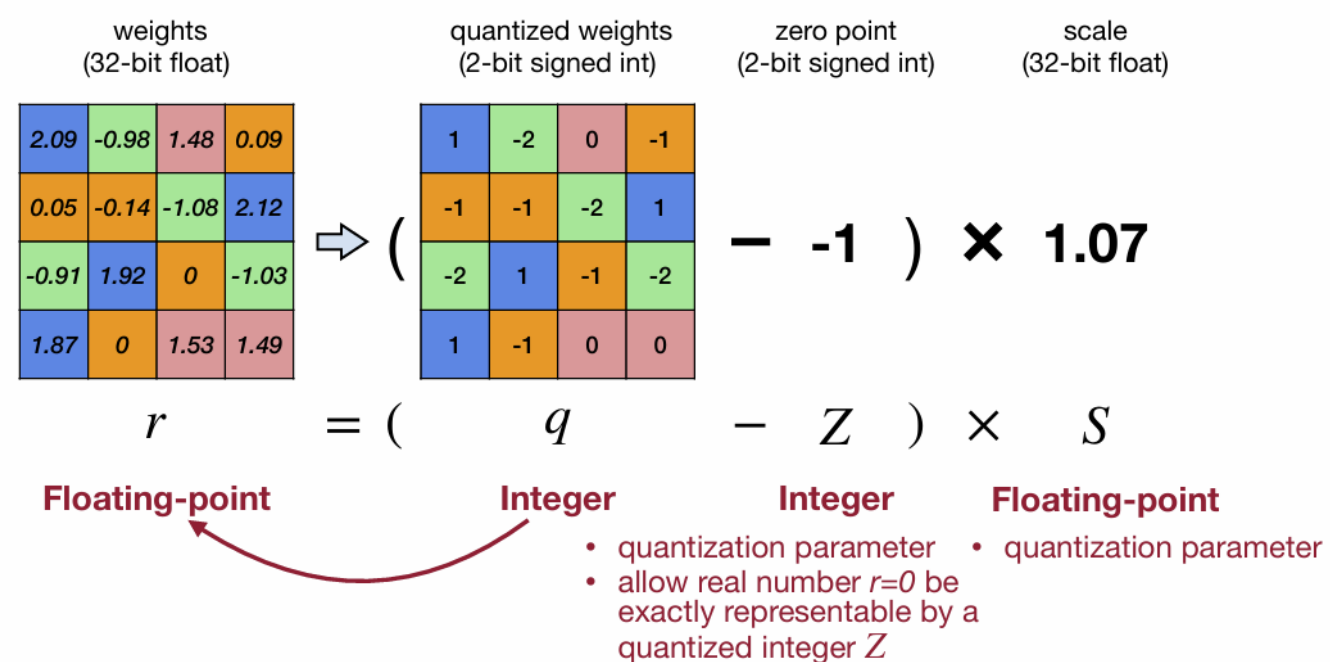
04 About Quantization – Method

• Linear quantization



Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

- 양자화 된 값과 실수 값 사이에 Scaling factor S
- 실수 0을 정확하게 표현하기 위해 Zero point Z
- S 와 Z 를 따로 저장
- 데이터의 형태와 계산 방식 모두 INT



04 About Quantization – Method

- Linear quantization

$$\mathbf{Y} = \mathbf{WX} + \mathbf{b}$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W (\mathbf{q}_W - Z_W) \cdot S_X (\mathbf{q}_X - Z_X) + S_b (\mathbf{q}_b - Z_b)$$

04 About Quantization – Method

- Linear quantization

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W (\mathbf{q}_W - Z_W) \cdot S_X (\mathbf{q}_X - Z_X) + S_b (\mathbf{q}_b - Z_b)$$



$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W \mathbf{q}_X + \mathbf{q}_{bias}) + Z_Y$$

04 About Quantization – Method

- Linear quantization

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W (\mathbf{q}_W - Z_W) \cdot S_X (\mathbf{q}_X - Z_X) + S_b (\mathbf{q}_b - Z_b)$$



$$Z_W = 0$$

$$Z_b = 0, \quad S_b = S_W S_X$$

$$\mathbf{q}_{bias} = \mathbf{q}_b - Z_X \mathbf{q}_W$$

Precompute!

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W \mathbf{q}_X + \mathbf{q}_{bias}) + Z_Y$$

04 About Quantization – Method

- Linear quantization

$$Y = \text{Conv}(W, X) + b$$

$Z_W = 0$
 $Z_b = 0, S_b = S_W S_X$
 $\downarrow \mathbf{q}_{bias} = \mathbf{q}_b - \text{Conv}(\mathbf{q}_W, Z_X)$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} \left(\text{Conv}(\mathbf{q}_W, \mathbf{q}_X) + \mathbf{q}_{bias} \right) + Z_Y$$

Rescale to N -bit Int N -bit Int Mult. 32-bit Int Add. N -bit Int Add

04 About Quantization – PTQ

- **During Training**

04 About Quantization – PTQ

- Adaptive Rounding

$$\tilde{w} = \lfloor \lfloor w \rfloor + \delta \rfloor, \delta \in [0, 1]$$

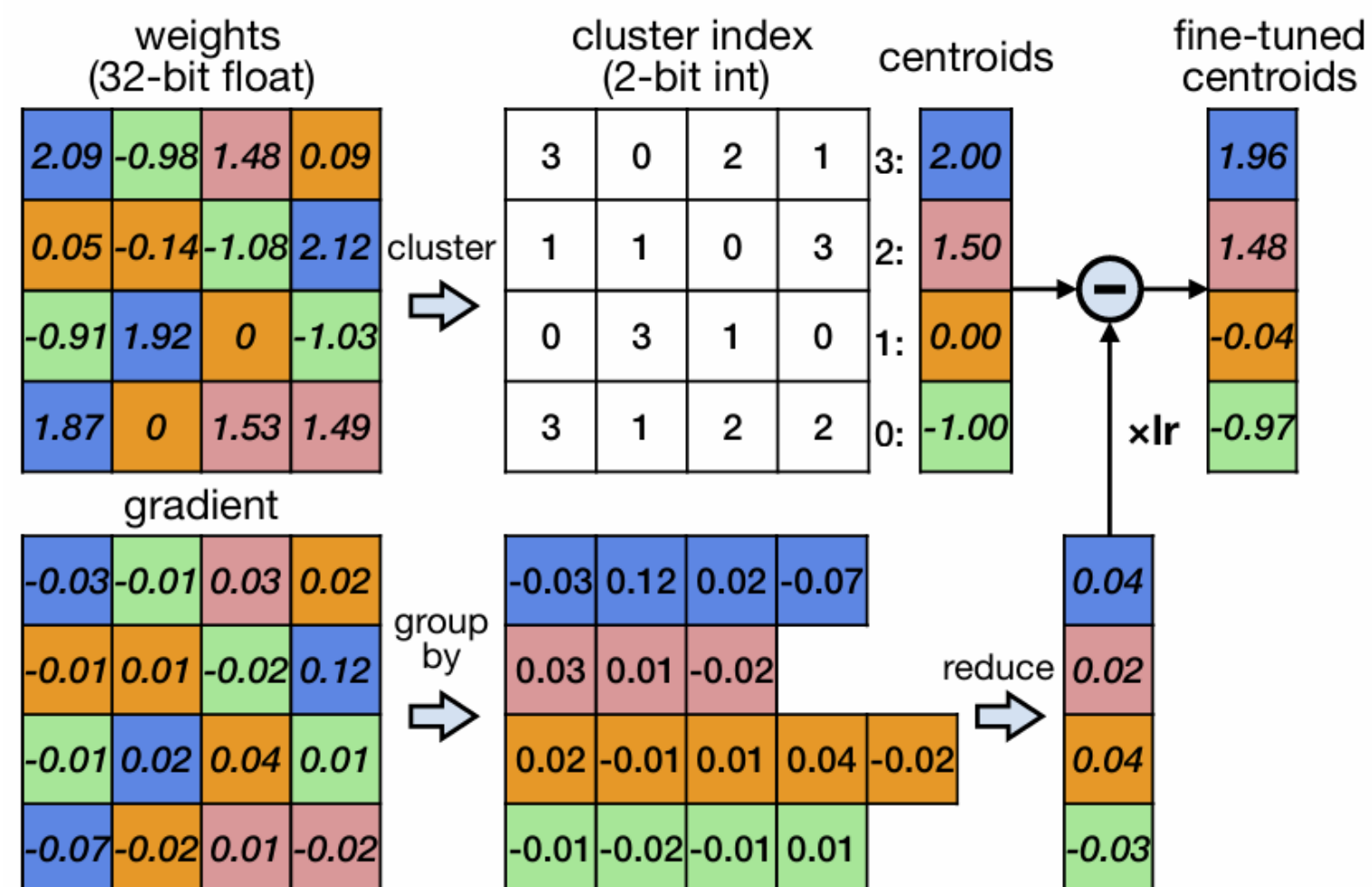
- 단순한 반올림 대신 loss를 더 줄일 수 있는 값을 학습
- δ 는 0과 1 중 하나이지만 학습이 가능하게 하기 위해 미분가능한 함수 $h(V)$ 를 사용
예) rectified Sigmoid

$$\begin{aligned} & \operatorname{argmin}_{\mathbf{V}} \|\mathbf{W}\mathbf{x} - \tilde{\mathbf{W}}\mathbf{x}\|_F^2 + \lambda f_{reg}(\mathbf{V}) \\ \rightarrow & \operatorname{argmin}_{\mathbf{V}} \|\mathbf{W}\mathbf{x} - \lfloor \lfloor \mathbf{W} \rfloor + \mathbf{h}(\mathbf{V}) \rfloor \mathbf{x} \|_F^2 + \lambda f_{reg}(\mathbf{V}) \end{aligned}$$

04 About Quantization – QAT

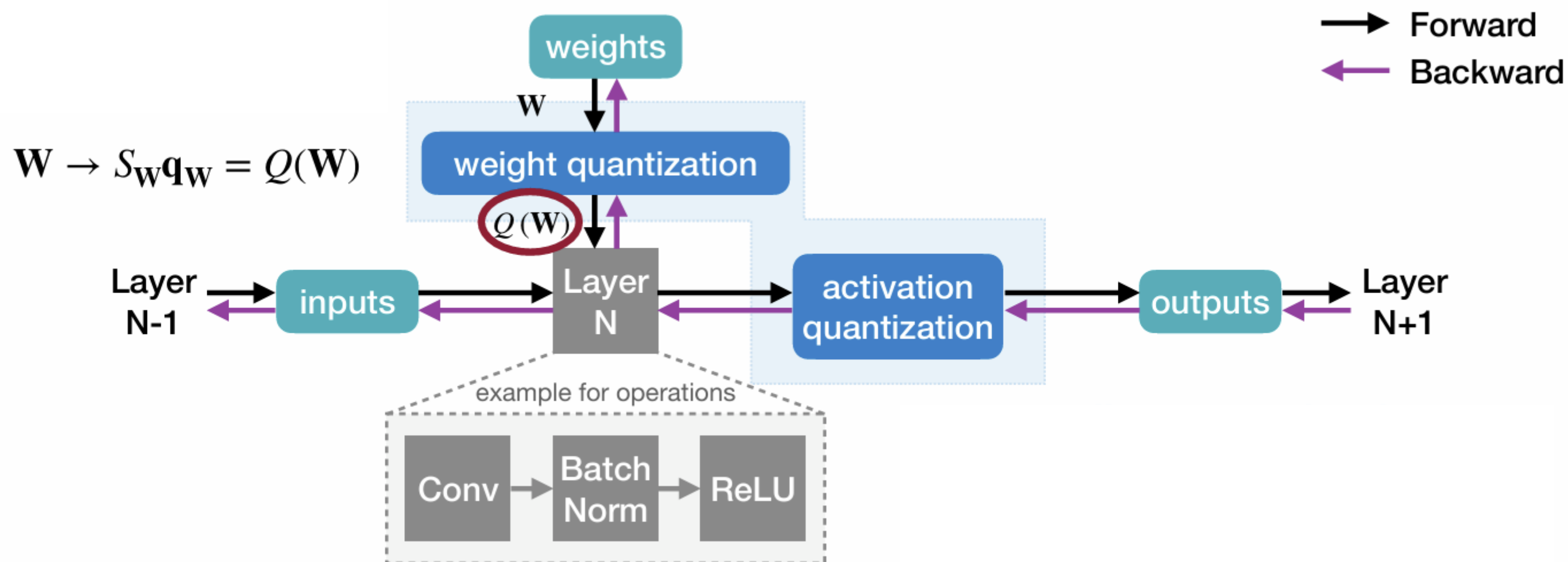
• Fine-tuning K-means based quantization

- 같은 cluster에 있는 weight 값의 gradient를 모두 더함
- 더한 값을 code book의 centroids 값을 update



04 About Quantization – QAT

- Simulated/Fake Quantization



05 Quantization Simulation

- K-means based quantization
 - Cluster 수: 32, 16, 12, 8 -> 5비트, 4비트, 4비트, 3비트
 - Granularity: Per-Tensor Quantization
 - Uniform Quantization
 - 대상: Convolution layer

Cluster 수	Accuracy
32	9394/10000 (93.9 %)
16	9341/10000 (93.4 %)
12	9267/10000 (92.7 %)
8	8938/10000 (89.4 %)

```
weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() == 4]
print(len(weight_param_list))

pruned_model = copy.deepcopy(model)
params_dict = dict(pruned_model.named_parameters())

for l in range(len(weight_param_list)):
    name, weight = weight_param_list[l]
    # print(name)
    # print(weight.shape)
    weight_tensor = weight.data.clone()

    X = weight_tensor.cpu().numpy().reshape(-1, 1)
    kmeans = KMeans(n_clusters=32, random_state=0).fit(X)

    labels = kmeans.labels_
    centers = kmeans.cluster_centers_

    new_weights = centers[labels].reshape(weight_tensor.shape)
    new_weight_tensor = torch.from_numpy(new_weights).to(DEVICE).type_as(weight_tensor)

    with torch.no_grad():
        params_dict[name].copy_(new_weight_tensor)

rcorrect, acc = mm.Test(pruned_model, test_DL, DEVICE)
print(f"Test accuracy: {rcorrect}/{len(test_DL.dataset)} ({acc} %)")
```

05 Quantization Simulation

- Linear quantization
 - 비트 수: 2~8 비트
 - Granularity: Per-Tensor Quantization
 - Uniform Quantization
 - 대상: Convolution layer

```
Test accuracy(2 비트): 1000/10000 (10.0 %)
Test accuracy(3 비트): 3948/10000 (39.5 %)
Test accuracy(4 비트): 9328/10000 (93.3 %)
Test accuracy(5 비트): 9364/10000 (93.6 %)
Test accuracy(6 비트): 9401/10000 (94.0 %)
Test accuracy(7 비트): 9410/10000 (94.1 %)
Test accuracy(8 비트): 9418/10000 (94.2 %)
```

```
weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() == 4]
print(len(weight_param_list))

for bit_num in range(2, 9):
    idx=bit_num-2
    q_max_=q_max[idx]
    q_min_=q_min[idx]

    quanted_model = copy.deepcopy(model)
    params_dict = dict(quanted_model.named_parameters())

    for l in range (len(weight_param_list)):

        name,weight=weight_param_list[l]
        # print(name)
        # print(weight.shape)

        weight_tensor=weight.data.clone()

        r_max=torch.max(weight_tensor)
        r_min=torch.min(weight_tensor)
        S_ = (r_max - r_min) / (q_max_ - q_min_)
        Z_=torch.round(q_min_ - r_min/S_)

        quanted_weight=torch.round(weight_tensor/S_+Z_)
        weight_dequant=(quanted_weight-Z_)*S_

        with torch.no_grad():
            params_dict[name].copy_(weight_dequant)

rcorrect,acc=mm.Test([quanted_model, test_DL, DEVICE])
print(f"Test accuracy({bit_num} 비트): {rcorrect}/{len(test_DL.dataset)} ({acc} %)")
```

05 Quantization Simulation

- Linear quantization
 - 비트 수: 2~8 비트
 - Granularity: Per-Channel Quantization
 - Uniform Quantization
 - 대상: Convolution layer

```
Test accuracy(2 비트): 1931/10000 (19.3 %)
Test accuracy(3 비트): 9116/10000 (91.2 %)
Test accuracy(4 비트): 9370/10000 (93.7 %)
Test accuracy(5 비트): 9413/10000 (94.1 %)
Test accuracy(6 비트): 9413/10000 (94.1 %)
Test accuracy(7 비트): 9416/10000 (94.2 %)
Test accuracy(8 비트): 9414/10000 (94.1 %)
```

```
weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() == 4]
print(len(weight_param_list))

for bit_num in range(2, 9):
    idx=bit_num-2
    q_max_=q_max[idx]
    q_min_=q_min[idx]

    quanted_model = copy.deepcopy(model)
    params_dict = dict(quanted_model.named_parameters())

    for l in range (len(weight_param_list)):

        name,weight=weight_param_list[l]
        # print(name)
        # print(weight.shape)

        weight_tensor=weight.data.clone()
        quanted_weight=torch.zeros_like(weight_tensor)
        weight_dequantized=torch.zeros_like(weight_tensor)

        for k in range(weight_tensor.shape[0]):
            r_max=torch.max(weight_tensor[k])
            r_min=torch.min(weight_tensor[k])
            S_ = (r_max - r_min) / (q_max_ - q_min_)
            Z_=torch.round(q_min_ - r_min/S_)

            quanted_weight[k]=torch.round(weight_tensor[k]/S_+Z_)
            weight_dequantized[k]=(quanted_weight[k]-Z_)*S_

        with torch.no_grad():
            params_dict[name].copy_(weight_dequantized)

    rcorrect,acc=mm.Test(quanted_model, test_DL, DEVICE)
    print(f"Test accuracy({bit_num} 비트): {rcorrect}/{len(test_DL.dataset)} ({acc} %)")
```


05 Quantization Simulation

- Linear quantization
 - 비트 수: 2~8 비트
 - Granularity: Per-Channel & row Quantization
 - Uniform Quantization
 - 대상: Convolution layer & FC layer

```
Test accuracy(2 비트): 1698/10000 (17.0 %)
Test accuracy(3 비트): 9114/10000 (91.1 %)
Test accuracy(4 비트): 9374/10000 (93.7 %)
Test accuracy(5 비트): 9415/10000 (94.2 %)
Test accuracy(6 비트): 9413/10000 (94.1 %)
Test accuracy(7 비트): 9416/10000 (94.2 %)
Test accuracy(8 비트): 9414/10000 (94.1 %)
```

```
weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0] and p[1].dim() > 1]
print(len(weight_param_list))

for bit_num in range(2, 9):
    idx=bit_num-2
    q_max_=q_max[idx]
    q_min_=q_min[idx]

    quanted_model = copy.deepcopy(model)
    params_dict = dict(quanted_model.named_parameters())

    for l in range (len(weight_param_list)):

        name,weight=weight_param_list[l]
        # print(name)
        # print(weight.shape)

        weight_tensor=weight.data.clone()
        quanted_weight=torch.zeros_like(weight_tensor)
        weight_dequantized=torch.zeros_like(weight_tensor)

        for k in range(weight_tensor.shape[0]):
            r_max=torch.max(weight_tensor[k])
            r_min=torch.min(weight_tensor[k])
            S_ = (r_max - r_min) / (q_max_ - q_min_)
            Z_=torch.round(q_min_ - r_min/S_)

            quanted_weight[k]=torch.round(weight_tensor[k]/S_+Z_)
            weight_dequantized[k]=(quanted_weight[k]-Z_)*S_

        with torch.no_grad():
            params_dict[name].copy_(weight_dequantized)

    rcorrect,acc=mm.Test(quanted_model, test_DL, DEVICE)
    print(f"Test accuracy({bit_num} 비트): {rcorrect}/{len(test_DL.dataset)} ({acc} %)")
```


05 Quantization Simulation

- Linear quantization – 연산 과정 구현
 - 비트 수: 2,5,8
 - Granularity: Per-Tensor
 - 대상: FC layer

비트 수	평균 오차
2	0.1094
5	0.0116
8	0.0014

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W (\mathbf{q}_W - Z_W) \cdot S_X (\mathbf{q}_X - Z_X) + S_b (\mathbf{q}_b - Z_b)$$

$$Z_W = 0$$

$$Z_b = 0, \quad S_b = S_W S_X$$

$$\mathbf{q}_{bias} = \mathbf{q}_b - Z_X \mathbf{q}_W \quad \text{Precompute!}$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W \mathbf{q}_X + \mathbf{q}_{bias}) + Z_Y$$

```
input_sub=torch.nn.Sequential(*list(model.features.children()),torch.nn.Flatten(start_dim=1),*list(model.classifier.children())[:3]).to(DEVICE)
output_sub=list(model.classifier.children())[3]
```

```
bit_width = list(range(2, 9))
q_min = [ -2**(b-1) for b in bit_width ]
q_max = [ 2**(b-1) - 1 for b in bit_width ]
```

```
bit_num=2
idx=bit_num-2
q_max_=q_max[idx]
q_min_=q_min[idx]

S_X_list=[]
S_Y_list=[]
Z_X_list=[]
Z_Y_list=[]

for images,_ in val_DL:
    input=input_sub(images.to(DEVICE))
    output=output_sub(input)

    r_x_max=input.amax(dim=1)
    r_x_min=input.amin(dim=1)
    S_x= (r_x_max - r_x_min) / (q_max_ - q_min_)
    Z_x=torch.round(q_min_ - r_x_min/S_x)

    r_y_max=output.amax(dim=1)
    r_y_min=output.amin(dim=1)
    S_y= (r_y_max - r_y_min) / (q_max_ - q_min_)
    Z_y=torch.round(q_min_ - r_y_min/S_y)

    S_X_list.append(S_x.mean().item())
    S_Y_list.append(S_y.mean().item())
    Z_X_list.append(Z_x.mean().item())
    Z_Y_list.append(Z_y.mean().item())
```

```
S_x=np.mean(S_X_list)
Z_x=np.mean(np.round(Z_X_list))
S_y=np.mean(S_Y_list)
Z_y=np.mean(np.round(Z_Y_list))
```

```
bias=list(model.classifier.children())[3].bias
weight=list(model.classifier.children())[3].weight
```

```
r_w_max=torch.max(torch.abs(weight))
r_b_max=torch.max(torch.abs(bias))
S_w= (r_w_max) / (q_max_ +1)
Z_w=0
S_b= S_w*S_x
Z_b=0
```

```
weight_tensor=weight.data.clone()
bias_tensor=bias.data.clone()
quanted_weight=torch.zeros_like(weight_tensor)
quanted_bias=torch.zeros_like(bias_tensor)
weight_dequanted=torch.zeros_like(weight_tensor)
bias_dequanted=torch.zeros_like(bias_tensor)
```

```
quanted_weight=torch.round(weight_tensor/S_w+Z_w)
weight_dequanted=(quanted_weight-Z_w)*S_w
quanted_bias=torch.round(bias_tensor/S_b+Z_b)
bias_dequanted=(quanted_bias-Z_b)*S_b
```

```
q_bias=quanted_bias - Z_x*quanted_weight.sum(dim=1)
```

```
sum=0
for images,_ in test_DL:
    input=input_sub(images.to(DEVICE))
    output=output_sub(input)

    q_x=torch.round(input/S_x+Z_x)
    q_y=torch.matmul(q_x,quanted_weight.T) + q_bias
    q_y=q_y*((S_w*S_x)/S_y) + Z_y

    dequanted_y=(q_y - Z_y)*S_y

    sum+=torch.abs(output-dequanted_y).mean().item()

print(f"Average error: {sum/len(test_DL):.4f}")
```


발표를 들어주셔서

감사합니다

